

Programmazione Competitiva

Antonio Ianniello

Indice

1	Complessità Computazionale	4
1.1	La notazione asintotica	4
1.1.1	Notazione asintotica O e caso peggiore	4
1.1.2	Notazione asintotica Ω e caso migliore	5
1.1.3	Gerarchia delle complessità asintotiche	6
1.1.4	Algebra della Notazione Asintotica	7
1.2	Complessità computazionale delle principali istruzioni	8
1.3	Esercizi di complessità computazionale	13
1.3.1	Esercizio 1	13
1.3.2	Esercizio 2	14
1.3.3	Esercizio 3	15
2	Strutture dati in C++	16
2.1	Vector	16
2.1.1	Iterazione tramite <code>begin()</code> e <code>end()</code>	16
2.1.2	Operazioni principali sui vector	17
2.1.3	Inserimento ed eliminazione in posizioni specifiche	18
2.1.4	Altre operazioni utili	19
2.1.5	Iterazione su un vector	19
2.1.6	Complessità computazionale riassunta	20
2.2	Map	21
2.2.1	Dichiarazione di una map	21
2.2.2	Operazioni principali sulle map	22
2.2.3	Altre operazioni utili	23
2.2.4	Iterazione su una map	23
2.2.5	Complessità computazionale riassunta	24
2.3	Set	25
2.3.1	Dichiarazione di un set	25
2.3.2	Operazioni principali sui set	25
2.3.3	Altre operazioni utili	28
2.3.4	Iterazione su un set	29
2.3.5	Esempio pratico	29
2.3.6	Complessità computazionale riassunta	30
2.4	Multiset	31
2.4.1	Caratteristiche principali	31
2.4.2	Esempio pratico	31
2.4.3	Complessità computazionale riassunta	32
2.5	Stack	33
2.5.1	Dichiarazione di uno stack	33
2.5.2	Operazioni principali sullo stack	34

2.5.3	Iterazione su uno stack	34
2.5.4	Esempio pratico	35
2.5.5	Complessità computazionale riassunta	35
2.6	Queue	36
2.6.1	Dichiarazione di una queue	36
2.6.2	Operazioni principali sulla queue	36
2.6.3	Iterazione su una queue	37
2.6.4	Esempio pratico	38
2.6.5	Complessità computazionale riassunta	38
2.7	Priority Queue	39
2.7.1	Dichiarazione di una priority_queue	39
2.7.2	Operazioni principali sulla priority_queue	40
2.7.3	Iterazione su una priority_queue	40
2.7.4	Esempio pratico	41
2.7.5	Complessità computazionale riassunta	41
2.8	Deque	42
2.8.1	Dichiarazione di una deque	42
2.8.2	Operazioni principali sulla deque	42
2.8.3	Iterazione su una deque	43
2.8.4	Esempio pratico	44
2.8.5	Complessità computazionale riassunta	45
2.8.6	Differenze tra queue e deque	45
3	Brute Force e Algoritmi Greedy	47
3.1	Algoritmi Brute Force	47
3.1.1	Antivirus	48
3.1.2	Ruota della fortuna	53
3.1.3	Stick Lengths	57
3.1.4	Nearest Smaller Values	60
3.2	Idee Greedy	63
3.2.1	Calcolatrice Rotta	65
3.2.2	Precise Average	68
4	Pattern Algoritmici	71
4.1	Sliding Window	71
4.1.1	Christmas Light	76
4.1.2	Playlist	78
4.1.3	Distinct Values Subarray	78
4.2	Bitmask	78
4.2.1	Apple Division	79
4.3	Prefix Sum	83
4.3.1	C - Index $\times$ A(Continuous ver.)	87

Introduzione

I Campionati Italiani di Informatica (ex Olimpiadi) sono una competizione rivolta agli studenti delle istituzioni scolastiche secondarie di II grado. Giunte ormai alla XXVI edizione, fanno parte del programma di valorizzazione delle eccellenze che la Direzione Generale per gli ordinamenti scolastici e la valutazione del sistema nazionale di istruzione del Ministero dell'Istruzione promuove e finanzia ogni anno.

La partecipazione è aperta alle classi I-IV di tutte le istituzioni scolastiche secondarie di II grado, statali e paritarie, che ritengano di avere studenti con potenziale interesse per l'informatica, soprattutto riguardo gli aspetti logici e algoritmici di tale disciplina. La partecipazione è adatta e consigliata anche a coloro che ancora non hanno studiato informatica a livello curricolare, per consentire agli studenti di interessarsi e avvicinarsi gradualmente alla materia.

Gli studenti potranno prepararsi alla prima fase tramite le prove degli anni passati disponibili sul sito `scolastiche.olinfo.it`.

Il regolamento, inclusa la descrizione delle varie fasi è riportato in appendice ??.

Si fa notare che durante la prima fase (selezione scolastica), verrà richiesta la soluzione dei problemi utilizzando esclusivamente **pseudocodice** le cui regole sono riportate in appendice ??

In questo documento verranno trattati argomenti teorici e pratici al fine di affrontare le olimpiadi di informatica con una degna preparazione.

Capitolo 1

Complessità Computazionale

La programmazione competitiva è un'attività di programmazione finalizzata alla partecipazione a gare organizzate in cui i partecipanti devono risolvere problemi algoritmici e matematici scrivendo programmi che soddisfano specifiche restrizioni di tempo e memoria.

È importante valutare la complessità di ogni algoritmo costruito in quanto può essere necessario limitare l'esecuzione ad un certo tempo (es. l'algoritmo deve terminare entro 2 secondi dallo start).

Nel contesto della competizione, consideriamo un algoritmo eseguito da un computer in grado di effettuare circa 10^8 operazioni al secondo. Sarà sufficiente calcolare il numero di operazioni richieste dall'algoritmo per determinare il tempo totale necessario per completarne l'esecuzione.

Ovviamente, calcolare il numero totale di operazioni potrebbe non essere semplice, poiché esso dipende sia dal numero di dati in input sia dall'efficienza "interna" degli algoritmi (che approfondiremo più avanti).

1.1 La notazione asintotica

Per valutare la complessità temporale di un algoritmo, occorre introdurre un concetto matematico importantissimo: **la notazione asintotica**.

In matematica, questa consente di confrontare il tasso di crescita (comportamento asintotico) di una funzione nei confronti di un'altra.

In informatica si sfrutta questo concetto per valutare di quanto aumenta il tempo di esecuzione di un algoritmo ($T(n)$) in relazione all'aumento dei dati in input (n).

Esistono 3 tipi di notazione asintotica:

- **Notazione asintotica O** , chiamata *limite superiore asintotico*
- **Notazione asintotica Ω** , chiamata *limite inferiore asintotico*
- **Notazione asintotica Θ** , chiamata *limite stretto asintotico*

Andiamo nel dettaglio di ognuna per capire come possono esserci utili

1.1.1 Notazione asintotica O e caso peggiore

Date due funzioni $T(n)$ e $f(n)$, si dice che $T(n) \in O(f(n))$ se esistono costanti positive c e n_0 tali che:

$$0 \leq T(n) \leq c \cdot f(n) \quad \text{per ogni } n \geq n_0$$

In altre parole, a partire da un certo valore di n , $f(n)$ rappresenta un limite superiore asintotico per la crescita di $T(n)$.

Informalmente, $T(n) \in O(f(n))$ se $T(n)$ **non cresce più velocemente di** $f(n)$. Approssimando la funzione $T(n)$ con $f(n)$ per $n > n_0$, si ottiene una **sovrastima** di $T(n)$.

In informatica, quando analizziamo un algoritmo, ci interessa generalmente il **caso peggiore**. Il tempo reale di esecuzione può variare a seconda dell'input, e calcolarlo esattamente per tutti i possibili input è spesso **molto difficile** o impossibile. Per questo motivo, sostituendo $T(n)$ con una funzione più semplice come $c \cdot f(n)$, che **sovrastima** il tempo reale, otteniamo una stima della complessità temporale valida in **ogni situazione possibile**.

In figura 1.1 è riportata la rappresentazione grafica di quanto detto. Sostituendo $T(n)$ con $c \cdot f(n)$, stiamo considerando il **caso peggiore** possibile, senza dover calcolare ogni possibile esecuzione dell'algoritmo.

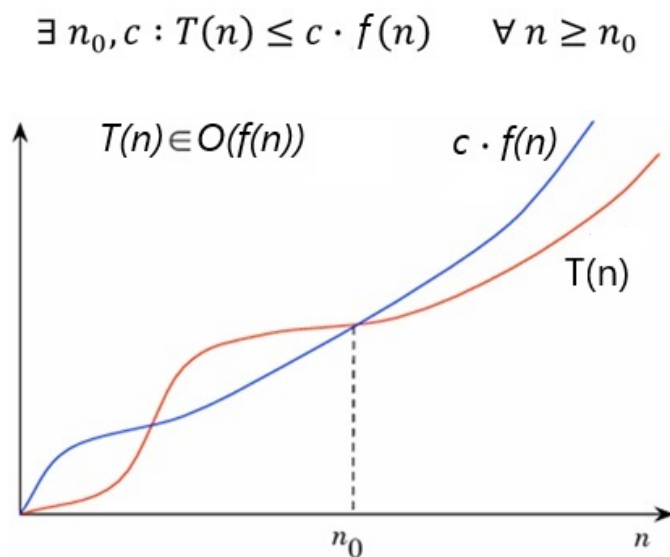


Figura 1.1: Rappresentazione grafica della notazione O e del caso peggiore

1.1.2 Notazione asintotica Ω e caso migliore

Date due funzioni $T(n)$ e $f(n)$, si dice che $T(n) \in \Omega(f(n))$ se esistono costanti positive c e n_0 tali che:

$$0 \leq c \cdot f(n) \leq T(n) \quad \text{per ogni } n \geq n_0$$

In altre parole, a partire da un certo valore di n , $f(n)$ rappresenta un limite inferiore asintotico per la crescita di $T(n)$.

Informalmente, $T(n) \in \Omega(f(n))$ se $T(n)$ **non cresce più lentamente di** $f(n)$. Approssimando la funzione $T(n)$ con $f(n)$ per $n > n_0$, si ottiene una **sottostima** di $T(n)$.

In informatica, il limite inferiore viene utilizzato per indicare il **caso migliore** di esecuzione di un algoritmo. Il tempo reale di esecuzione può variare a seconda dell'input, ma sostituendo $T(n)$ con una funzione semplice come $c \cdot f(n)$ che **sottostima** il tempo reale, otteniamo una stima valida almeno quanto il tempo minimo richiesto dall'algoritmo.

In figura 1.2 è riportata la rappresentazione grafica di quanto detto. Sostituendo $T(n)$ con $c \cdot f(n)$ in questo contesto, stiamo considerando il **caso migliore** possibile.

$$\exists n_0, c : T(n) \geq c \cdot f(n) \quad \forall n \geq n_0$$

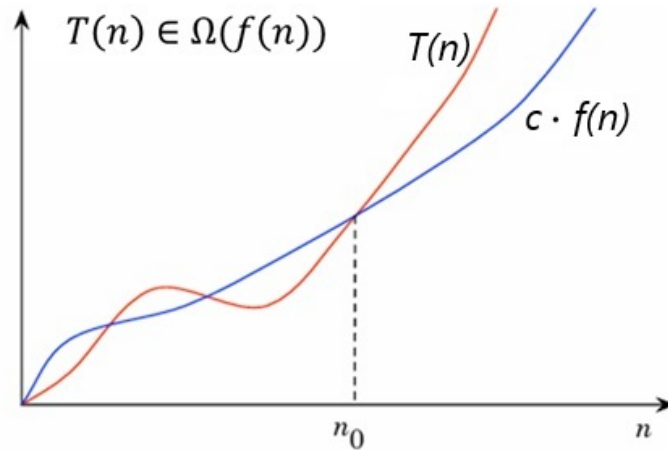


Figura 1.2: Rappresentazione grafica della notazione Ω e del caso migliore

1.1.3 Gerarchia delle complessità asintotiche

Per avere un'idea chiara della velocità di crescita delle principali funzioni, possiamo organizzare le complessità asintotiche in una tabella. La tabella 1.1 mostra le funzioni più comuni ordinate in **ordine crescente di crescita**, partendo dalle più lente fino a quelle che crescono più rapidamente.

Ordine di crescita	Funzioni tipiche
Costante	$O(1)$
Logaritmica	$O(\log n)$
Radicale	$O(\sqrt{n})$
Lineare	$O(n)$
Lineare-logaritmica	$O(n \log n)$
Quadratica	$O(n^2)$
Cubica	$O(n^3)$
Polinomiale	$O(n^k), k > 3$
Esponenziale	$O(2^n), O(k^n)$
Fattoriale	$O(n!)$
Super-esponenziale	$O(n^n)$

Tabella 1.1: Funzioni comuni ordinate per crescita asintotica

1.1.4 Algebra della Notazione Asintotica

Per semplificare il calcolo del costo computazionale asintotico degli algoritmi si possono sfruttare delle semplici regole che enunciamo senza riportarne la dimostrazione¹.

Prima regola: moltiplicazione per costante

Se esiste una funzione $T(n) \geq 0$ tale che:

$$T(n) \in O(f(n)),$$

allora, per ogni costante positiva $k > 0$, vale:

$$k \cdot T(n) \in O(f(n)).$$

Informalmente, possiamo enunciare questa regola dicendo che: **le costanti moltiplicative si possono ignorare.**

Seconda regola: somma

Siano $T(n) \geq 0$ e $S(n) \geq 0$ due funzioni tali che:

$$T(n) \in O(f(n)) \quad \text{e} \quad S(n) \in O(g(n)).$$

Allora vale:

$$T(n) + S(n) \in O(f(n) + g(n)) = O(\max(f(n), g(n))).$$

In altre parole:

Se $f(n)$ cresce più velocemente di $g(n)$, allora $O(f(n) + g(n)) = O(f(n))$.

Se $g(n)$ cresce più velocemente di $f(n)$, allora $O(f(n) + g(n)) = O(g(n))$.

In termini ancora più semplici, possiamo affermare che in una somma si può **trascurare** il termine più piccolo.

Terza regola: prodotto

Siano $T(n) \geq 0$ e $S(n) \geq 0$ due funzioni tali che:

$$T(n) \in O(f(n)) \quad \text{e} \quad S(n) \in O(g(n)).$$

Allora vale:

$$T(n) \cdot S(n) \in O(f(n) \cdot g(n)).$$

MOLTO informalmente, possiamo scrivere che:

$$O(f(n)) \cdot O(g(n)) = O(f(n) \cdot g(n))$$

¹Le regole che verranno riportate sono valide sia per il caso peggiore che per il caso migliore. Visto che nei problemi informatici siamo interessati quasi sempre al caso peggiore, riporteremo l'algebra della notazione asintotica **O**

1.2 Complessità computazionale delle principali istruzioni

Operazioni elementari

Tutte le operazioni elementari, ovvero:

- assegnazione di una variabile,
- somma, sottrazione, moltiplicazione, divisione,
- confronto ($<$, $>$, $==$, $!=$, ...),
- accesso all'elemento di un array con indice noto,
- input o output di una singola variabile,

hanno sempre complessità computazionale $O(1)$.

Consideriamo il seguente frammento di codice C++:

```
1  int a = 5;
2  int b = 10;
3  int c;
4  int d;
5  int e;
6
7  if (a < b) {
8      c = a + b;
9      d = c * 2;
10 }
11 else {
12     c = a - b;
13     d = c * 3;
14     e = d + 1;
15 }
```

Analizziamo il costo computazionale passo per passo:

- `int a = 5;` $\rightarrow O(1)$
- `int b = 10;` $\rightarrow O(1)$
- `if (a < b)` $\rightarrow O(1)$ (un confronto tra due variabili)

A questo punto, il programma può entrare nel ramo `if` oppure nel ramo `else`:

- Ramo `if`: due operazioni elementari $\Rightarrow 2 \cdot O(1) = O(1)$
- Ramo `else`: tre operazioni elementari $\Rightarrow 3 \cdot O(1) = O(1)$

Poiché i due rami sono alternativi (solo uno viene eseguito), si considera il ramo con complessità maggiore, in questo caso il ramo `else`.

La complessità totale del frammento è quindi:

$$O(1) + O(1) + O(1) + O(1) = O(1)$$

In conclusione, l'intero blocco ha complessità **costante**, indipendentemente dal valore delle variabili: il numero di operazioni non dipende da nessun parametro in ingresso.

Complessità computazionale dei cicli

Ciclo for

Consideriamo il seguente frammento di codice C++:

```
1  for (int i = 0; i < n; i++) {  
2      somma += i;  
3  }
```

Analizziamo il comportamento del ciclo:

- L'inizializzazione `int i = 0` ha costo $O(1)$.
- La condizione `i < n` viene valutata $n + 1$ volte $\Rightarrow O(n + 1) = O(n)$.
- L'incremento `i++` viene eseguito n volte $\Rightarrow O(n)$.
- Il corpo del ciclo `somma += i;` ha costo $O(1)$ per iterazione, quindi n iterazioni $\Rightarrow O(n)$.

Sommando i costi:

$$O(1) + O(n) + O(n) + O(n) = O(n)$$

Il ciclo `for` ha dunque complessità complessiva $O(n)$, ovvero cresce linearmente con il numero di iterazioni.

Ciclo while

Il ciclo `while` ha un comportamento simile, ma il numero di iterazioni può dipendere da una condizione variabile. Esempio:

```
1  int i = 1;  
2  while (i < n) {  
3      i = i * 2;  
4  }
```

In questo caso, la variabile `i` raddoppia ad ogni iterazione, quindi il numero di iterazioni totali è pari al numero di volte in cui possiamo moltiplicare per 2 prima di superare n .

$$i = 1 \Rightarrow 2 \Rightarrow 4 \Rightarrow 8 \Rightarrow \dots \Rightarrow n$$

Il ciclo termina quando $2^k \geq n$, quindi $k = \log_2 n$. Di conseguenza, la complessità complessiva è:

$$O(\log n)$$

In generale, per un ciclo `while`, la complessità dipende dal numero di iterazioni necessarie affinché la condizione diventi falsa.

Costrutti Misti e/o Annidati

Quando i cicli sono combinati o condizionati tra loro, la complessità si calcola sommando o moltiplicando i costi dei vari blocchi, a seconda della struttura.

Esempio 1 — Cicli sequenziali:

```
1  for (int i = 0; i < n; i++) {  
2      // O(1)  
3  }  
4  for (int j = 0; j < n; j++) {  
5      // O(1)  
6  }
```

Le due sezioni non sono annidate ma consecutive, quindi:

$$T(n) = O(n) + O(n) = O(n)$$

Esempio 2 — Cicli annidati e condizione interna:

```
1  for (int i = 0; i < n; i++) {  
2      if (i % 2 == 0) {  
3          for (int j = 0; j < n; j++) {  
4              // O(1)  
5          }  
6      }  
7  }
```

Il ciclo interno si esegue solo per metà delle iterazioni esterne, ma l'ordine di grandezza resta invariato:

$$T(n) = O\left(\frac{n}{2} \times n\right) = O(n^2)$$

Algoritmi con Procedure non ricorsive

La procedura generale che dev'essere seguita per il calcolo della complessità computazionale consiste nei seguenti passi:

- si esamina il programma scendendo all'interno di ciascuna procedura o funzione fino ad individuare quella o quelle che non presentano chiamate ad altre procedure o funzioni al loro interno;
- per le procedure o funzioni che non contengono al loro interno chiamate ad altre procedure o funzioni, devono essere tenute presenti le considerazioni fatte fino ad ora e cioè applicare le regole sui singoli costrutti, in modo da determinare la complessità computazionale delle procedure o funzioni considerate come se fossero semplici blocchi di codice;
- determinata la complessità delle procedure/funzioni al punto precedente si passa a considerare invece la chiamata a queste ultime da parte di altre procedure/funzioni. Per ciascuna di tali procedure e funzioni viene calcolata la complessità computazionale, sostituendo a ciascuna chiamata di procedura o funzione la relativa complessità computazionale;
- si risale all'interno di questa pila di chiamate, ricordando la complessità computazionale calcolata ad ogni livello di risalita. Tale procedimento termina quando è stata calcolata la complessità di tutte le procedure e funzioni presenti nel programma principale;

- la complessità computazionale del *main()* si calcola sostituendo a ciascuna chiamata di procedura o funzione la relativa complessità computazionale, calcolata secondo le regole descritte nei passi precedenti.

Ad esempio, si consideri il seguente programma:

```

1  #include <iostream>
2  using namespace std;
3
4  void stampaVettore(int A[], int n) {
5      for (int i = 0; i < n; i++)
6          cout << A[i] << " ";          // O(1)
7  }
8
9  int sommaVettore(int A[], int n) {
10     int somma = 0;                      // O(1)
11     for (int i = 0; i < n; i++)          // n iterazioni
12         somma += A[i];                  // O(1) per iterazione
13     return somma;                      // O(1)
14 }
15
16 int main() {
17     int n = 100;                        // O(1)
18     int A[100];                         // O(1)
19
20     for (int i = 0; i < n; i++)          // n iterazioni
21         A[i] = i;                      // O(1) per iterazione
22
23     stampaVettore(A, n);                 // O(n)
24     int risultato = sommaVettore(A, n); // O(n)
25
26     cout << "Somma: " << risultato;    // O(1)
27     return 0;
28 }

```

1. La funzione `stampaVettore()` contiene un ciclo `for` che scorre n elementi, quindi:

$$T_{\text{stampaVettore}}(n) = O(n)$$

2. La funzione `sommaVettore()` ha anch'essa un ciclo su n elementi, con operazioni elementari all'interno:

$$T_{\text{sommaVettore}}(n) = O(n)$$

3. La funzione `main()` esegue:

- Un ciclo `for` che inizializza un vettore di n elementi $\rightarrow O(n)$.
- Una chiamata a `stampaVettore()` $\rightarrow O(n)$.
- Una chiamata a `sommaVettore()` $\rightarrow O(n)$.

Applicando la regola della somma:

$$T_{\text{main}}(n) = O(n) + O(n) + O(n) = O(n)$$

Algoritmi con Funzioni Ricorsive

Per calcolare la complessità di un algoritmo ricorsivo non è possibile utilizzare le tecniche che abbiamo già analizzato, ma è necessario introdurre le **relazioni di ricorrenza**; esse descrivono una funzione nei termini del suo valore su input sempre più piccoli.

Ad esempio, consideriamo la funzione classica per calcolare la somma dei primi n numeri:

```
1  #include <iostream>
2  using namespace std;
3
4  // Funzione ricorsiva
5  int somma(int n) {
6      if (n == 0) return 0;      // caso base O(1)
7      return n + somma(n - 1);  // ricorsione
8  }
9
10 int main() {
11     int n = 100;
12     int risultato = somma(n);
13     cout << "Somma: " << risultato;
14     return 0;
15 }
```

Analizzando la complessità, si nota che:

- La funzione `somma()` chiama se stessa una volta per ogni valore di n fino al caso base:

$$T(n) = T(n - 1) + O(1)$$

- Espandendo passo passo:

$$T(n) = O(1) + O(1) + \dots + O(1) \text{ (per } n \text{ volte)} = O(n)$$

- La funzione `main()` esegue:
 - Assegnazione di $n \rightarrow O(1)$
 - Chiamata a `somma(n)` $\rightarrow O(n)$
 - Stampa del risultato $\rightarrow O(1)$

Applicando la regola della somma:

$$T_{\text{main}}(n) = O(1) + O(n) + O(1) = O(n)$$

Consideriamo un altro esempio: la versione ricorsiva della funzione di Fibonacci:

```
1  int fib(int n) {
2      if (n <= 1) return n;
3      return fib(n-1) + fib(n-2);
4  }
```

Possiamo affermare che:

- La funzione chiama se stessa due volte per ogni valore di n fino al caso base:

$$T(n) = T(n-1) + T(n-2) + O(1)$$

- Questa ricorrenza cresce esponenzialmente, poiché ciascuna chiamata produce due nuove chiamate. Di conseguenza:

$$T(n) = O(2^n)$$

1.3 Esercizi di complessità computazionale

1.3.1 Esercizio 1

Considera il seguente codice C++:

```

1  int a = 5, b = 10;
2  int maxVal;
3
4  if (a > b) {
5      maxVal = a;
6      b = b + 1;
7  } else {
8      maxVal = b;
9      a = a + 1;
10     b = b + 2;
11 }
```

Domanda: Calcola la complessità computazionale totale di questo frammento di codice in termini di O .

SOLUZIONE

Analizziamo il frammento di codice riga per riga:

- `int a = 5, b = 10;` → $O(1)$ (assegnazione di variabili)
- `int maxVal;` → $O(1)$ (assegnazione di variabile)
- `if (a > b)` → $O(1)$ (confronto)

Ramo vero:

- `maxVal = a;` → $O(1)$
- `b = b + 1;` → $O(1)$

Ramo falso:

- `maxVal = b;` → $O(1)$
- `a = a + 1;` → $O(1)$
- `b = b + 2;` → $O(1)$

Somma complessiva: Ogni ramo contiene un numero costante di operazioni $O(1)$. Applicando la regola della somma:

$$T(n) = O(1) + O(1) + \max(O(1) + O(1), O(1) + O(1) + O(1)) = O(1)$$

Conclusione: Il frammento di codice ha complessità totale $O(1)$.

1.3.2 Esercizio 2

Considera il seguente frammento di codice in C++:

```
1  int sum = 0;
2  for (int i = 0; i < n; i++) {
3      if (i % 2 == 0) {
4          sum += i;
5      } else {
6          sum -= i;
7      }
8  }
```

Domanda: Determinare la complessità computazionale del frammento di codice sopra in termini di n . Considerando un dispositivo capace di fare 10^8 operazioni al secondo e considerando un input $n < 10^4$, il tempo di esecuzione dell'algoritmo è inferiore a 1 secondo?

SOLUZIONE

Analizziamo ancora una volta il frammento di codice riga per riga:

- `int sum = 0` $\rightarrow O(1)$
- Il ciclo `for(int i = 0; i < n; i++)` viene eseguito n volte:
quindi il ciclo contribuisce con $n \cdot T_{\text{interno}}$.
- All'interno del ciclo, abbiamo un `if-else`:
 - Condizione `i % 2 == 0` $\rightarrow O(1)$
 - Operazioni nel ramo vero `sum += i` $\rightarrow O(1)$
 - Operazioni nel ramo falso `sum -= i` $\rightarrow O(1)$

Quindi ogni iterazione del ciclo ha complessità:

$$T_{\text{iterazione}} = O(1) + O(1) = O(1)$$

- Moltiplicando per il numero di iterazioni:

$$T_{\text{ciclo}} = n \cdot O(1) = O(n)$$

- Sommando tutte le parti del codice:

$$T_{\text{totale}} = O(1) + O(n) = O(n)$$

Conclusion: La complessità complessiva del frammento di codice è $O(n)$.

$$t_{\text{esecuzione}} \approx \frac{n}{10^8} < \frac{10^4}{10^8} = 10^{-4} \text{ secondi}$$

Poiché $10^{-4} \ll 1$, possiamo concludere che l'algoritmo termina molto prima di 1 secondo.

1.3.3 Esercizio 3

Consideriamo il seguente codice C++:

```
1  int sum = 0;
2  for (int i = 0; i < n; i++) {
3      for (int j = 0; j < m; j++) {
4          sum += i + j;
5      }
6  }
```

Domanda: Determinare la complessità computazionale del frammento di codice sopra in termini di n . Considerando un dispositivo capace di fare 10^8 operazioni al secondo e considerando un input $n < 10^4$ e $m < 10^3$, il tempo di esecuzione dell'algoritmo è inferiore a 1 secondo?

Analisi della complessità:

- $\text{int sum} = 0 \rightarrow O(1)$
- Il ciclo esterno `for(int i = 0; i < n; i++)` viene eseguito n volte: quindi il ciclo contribuisce con $n \cdot T_{\text{interno}}$.
- Il ciclo interno `for(int j = 0; j < m; j++)` viene eseguito m volte: quindi la complessità è $m \cdot T_{\text{interno}}$
- Nel ciclo interno abbiamo un'operazione elementare: `sum += i+j` $\rightarrow O(1)$.
- Applicando la regola del prodotto per i cicli annidati:

$$T_{\text{ciclo}}(n, m) = n \cdot (m \cdot O(1)) = O(n \cdot m)$$

- Sommando l'assegnazione iniziale:

$$T_{\text{totale}}(n, m) = O(1) + O(n \cdot m) = O(n \cdot m)$$

Conclusione: La complessità complessiva del frammento di codice è

$$T(n, m) = O(n \cdot m)$$

Il tempo di esecuzione è:

$$t_{\text{esecuzione}} \approx \frac{n \cdot m}{10^8} < \frac{10^4 \cdot 10^3}{10^8} = \frac{10^7}{10^8} = 0.1 \text{ secondo}$$

Da cui ne deriva che il tempo di esecuzione è meno di 1 secondo (ma non di molto).

Capitolo 2

Strutture dati in C++

2.1 Vector

Il **vector** è una struttura dati della Standard Template Library (STL) che rappresenta un **array dinamico**. A differenza degli array statici, la sua dimensione può crescere o ridursi durante l'esecuzione del programma. È contenitore sequenziale, quindi mantiene l'ordine degli elementi inseriti.

Dichiarazione di un vector:

```
1  #include <vector>
2  #include <iostream>
3  using namespace std;
4
5  vector<int> v;           // vector vuoto di interi
6
7  vector<int> v2(10);      // vector di 10 elementi
   inizializzati a 0
8
9  vector<int> v3(5, 7);    // vector di 5 elementi inizializzati
   a 7
```

Prima di andare ai metodi di questa struttura, è opportuno spendere due parole sugli iteratori.

2.1.1 Iterazione tramite `begin()` e `end()`

I metodi `begin()` ed `end()` sono due funzioni fondamentali dei contenitori della STL, come **vector**, e servono per ottenere **iteratori** che consentono di scorrere gli elementi del contenitore.

- `v.begin()` Restituisce un iteratore che punta al **primo elemento** del **vector**. È come un "puntatore" all'inizio della sequenza di dati.
- `v.end()` Restituisce un iteratore che punta **alla posizione successiva all'ultimo elemento** del **vector**. Non rappresenta quindi un elemento valido, ma serve per **indicare la fine** del contenitore.

L'uso tipico di `begin()` e `end()` è all'interno di un ciclo, per visitare tutti gli elementi del **vector**:

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main() {
6      vector<int> v = {10, 20, 30, 40};
7
8      for (auto it = v.begin(); it != v.end(); ++it) {
9          cout << *it << " ";
10     }
11     return 0;
12 }

```

In questo esempio:

- `auto it = v.begin();` inizializza l'iteratore al primo elemento^{1 2};
- il ciclo prosegue finché `it` non raggiunge `v.end()`, cioè finché ci sono elementi validi;
- `*it` consente di accedere al valore dell'elemento puntato.

Nota: `v.end()` non punta a un elemento valido, quindi non bisogna mai dereferenziarlo (cioè non fare `*v.end()`), poiché ciò porterebbe a un comportamento indefinito.

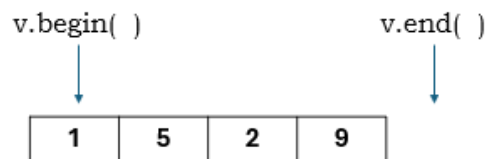


Figura 2.1: Iteratori `v.begin()` e `v.end()`

2.1.2 Operazioni principali sui vector

- **Accesso agli elementi:** `v[i]`

```

1  int x = v[1];           // accesso diretto           (senza
    cout << x;             // stampa l'elemento di v all'   controllo)
2  // stampa l'elemento di v all'   indice
    1

```

Complessità: $O(1)$.

- **Aggiunta in coda:** `push_back(x)`

```

1  v.push_back(3); // aggiunge 3 alla fine

```

Complessità: $O(1)$.

¹È stato usato un **auto it** in quanto `it` viene confrontato con l'iteratore `v.end()` ed è opportuno che questi siano dello stesso tipo.

²In alternativa, avrei potuto usare `vector<int>::iterator it`

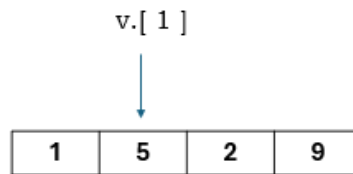


Figura 2.2: Accesso ad un elemento del vettore

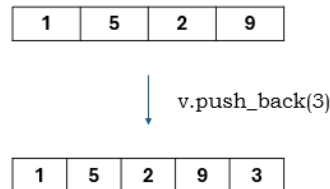


Figura 2.3: Inserimento di un elemento alla fine del vector

- **Rimozione dall'ultima posizione: `pop_back()`**

```
1 v.pop_back(); // rimuove l'ultimo elemento
```

Complessità: $O(1)$.

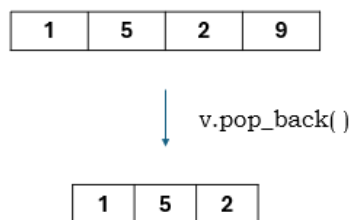


Figura 2.4: Cancellazione dell'ultimo elemento del vettore

- **Dimensione: `v.size()`** Restituisce il numero di elementi presenti nel vector.

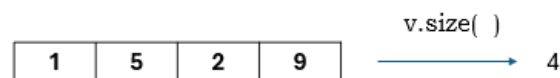


Figura 2.5: Numero di elementi nel vettore

- **Controllo se vuoto: `v.empty()`** Restituisce `true` se il vector non contiene elementi.

2.1.3 Inserimento ed eliminazione in posizioni specifiche

```
1 v.insert(v.begin() + 2, 10); // inserisce 10 in posizione 2 e
                              // shifta a destra tutti gli altri elementi
2
3 v.erase(v.begin() + 2);    // elimina elemento in posizione 2 e
                              // shifta a sinistra tutti gli altri
                              // elementi
```

Note:

- Inserimenti o cancellazioni in mezzo al vector hanno complessità $O(n)$, perché gli elementi successivi devono essere spostati.

2.1.4 Altre operazioni utili

- `v.clear()` → svuota il vector, $O(n)$
- `v.front()` → accede al primo elemento, $O(1)$
- `v.back()` → accede all'ultimo elemento, $O(1)$
- `v.resize(k)` → cambia la dimensione a k elementi (elimina quelli di troppo)

2.1.5 Iterazione su un vector

```

1 // Utilizzo di un ciclo for tradizionale
2 for (size_t i = 0; i < v.size(); i++) {
3     cout << v[i] << " ";
4 }
5
6 // Utilizzo del range-based for (C++11+)
7 for (int x : v) {
8     cout << x << " ";
9 }
10
11 // Utilizzo di iteratori
12 for (vector<int>::iterator it = v.begin(); it != v.end(); ++it) {
13     cout << *it << " ";
14 }

```

Note:

- la variabile di controllo del `for` è stata dichiarata come tipo `size_t`. Questo perché verrà confrontata con `v.size()` che, di fatto, è anch'essa di tipo `size_t`.
- `int x : v` è una sintassi valida da C++11 in poi. Molto semplicemente, `x` assume il valore di ogni elemento del vector `v`. In altre parole `x` è una copia e, come tale, non fa cambiare il valore degli elementi di `v` in caso scrivessi `x = 2`.

2.1.6 Complessità computazionale riassunta

Operazione	Complessità
Accesso a un elemento <code>v[i]</code>	$O(1)$
Inserimento in mezzo <code>v.insert(iteratore, valore)</code>	$O(n)$
Cancellazione in mezzo <code>v.erase(iteratore)</code>	$O(n)$
Aggiunta in coda <code>v.push_back(valore)</code>	$O(1)$
Eliminazione ultimo elemento <code>v.pop_back()</code>	$O(1)$
Dimensione <code>v.size()</code>	$O(1)$

Tabella 2.1: Complessità operazioni principali sui vector

2.2 Map

La **map** è una struttura dati della Standard Template Library (STL) che rappresenta un **contenitore associativo ordinato**. Permette di memorizzare coppie chiave-valore e garantisce che le chiavi siano **uniche** e **automaticamente ordinate**. Per accedere al valore, quindi, ho bisogno della chiave.

In figura 2.6 si può notare come le chiavi (`int`) siano ordinate in ordine crescente e siano associate a dei valori (`string`)

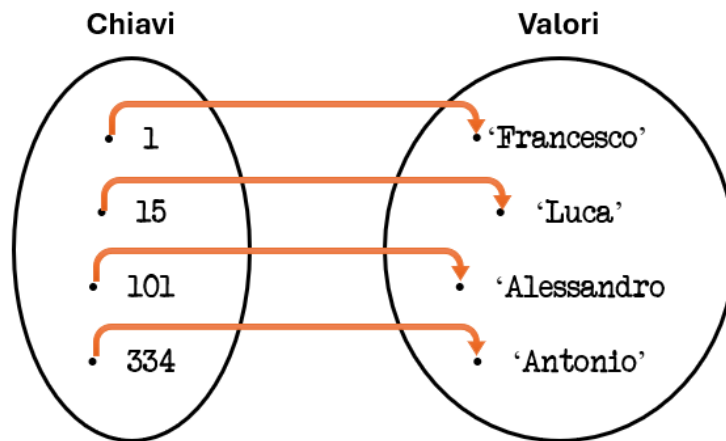


Figura 2.6: Rappresentazione grafica di una **map**

2.2.1 Dichiarazione di una map

```
1  #include <map>
2  #include <iostream>
3  using namespace std;
4
5  map<string, int> m1;           // mappa con chiavi stringa e valori
    interi
6
7  map<int, string> m2 = {       // mappa inizializzata con le coppie
                                // chiave-valore (le coppie saranno
                                // ordinate in ordine di chiave
                                // crescente automaticamente)
8      {101, "Alessandro"},
9      {334, "Antonio"},
10     {1, "Francesco"},
11     {15, "Luca"}
12 };
13
14 map<int, string, greater<int>> m3 = {
15     {1, "uno"},
16     {3, "tre"},
17     {2, "due"}
18 }; // greater<int> indica alla mappa di ordinare le chiavi dalla
    piu' grande al piu' piccola
```

Note:

- Da notare che nella riga 7 del codice, è stata inizializzata una mappa inserendo delle coppie chiave-valore non ordinate.
Sarà la struttura stessa ad ordinarle in base alle chiavi, in ordine crescente.
- Nella riga 14, invece, è stato fatto uso di **greater<int>** che ordina la mappa per chiavi decrescenti (se la chiave fosse stata **string** avrei dovuto usare **greater<string>**)

In un certo senso, si può vedere una map come un array associativo dinamico: invece di accedere agli elementi tramite un indice numerico, si accede tramite una chiave qualsiasi (int o string, ecc)

2.2.2 Operazioni principali sulle map

Le operazioni che seguono fanno riferimento alla mappa della figura 2.6.

- **Accesso ad un elemento** tramite chiave

```
1      cout << m[1];           // stampa "Francesco"
```

Complessità: $O(\log n)$

Nota: se la chiave non esiste, l'operatore `[]` crea automaticamente un nuovo elemento con valore di default.

- **Inserimento di un elemento:** `m.insert` o assegnamento tramite chiave

```
1      m[3] = "Luigi";           // inserisce la coppia  
                                     (3, "Luigi")  
2      m.insert({2, "Anna"});    // inserisce la coppia (2,  
                                     "Anna")
```

Complessità: $O(\log n)$ per ciascun inserimento. Bisogna infatti ricordare che le mappe sono ordinate e, quindi l'inserimento va messo in posizione corretta.

Nota: l'`insert` viene ignorato se si prova ad inserire un valore attraverso una chiave già esistente.

- **Rimozione di un elemento:** `erase`

```
1      m.erase(1);              // rimuove la chiave 1 e  
                                     il relativo valore
```

Complessità: $O(\log n)$

- **Dimensione:** `m.size()` Restituisce il numero di coppie chiave-valore nella mappa. Nel caso della figura 2.6:

```
1      cout << m.size();        // stampa 4
```

Complessità: $O(1)$

- **Cercare un elemento:** `m.find(key)`

Questo metodo cerca la chiave **key** nella mappa. Se la trova restituisce un iteratore alla coppia, altrimenti restituisce `m.end()`.

```

1      #include <iostream>
2      #include <map>
3      using namespace std;
4
5      int main() {
6          map<int, string> m = {
7              {1, "uno"},
8              {2, "due"},
9              {3, "tre"}
10         };
11
12         int chiave = 2;
13
14         // m.find(chiave) restituisce un iteratore alla coppia
15         // (chiave, valore) se la chiave esiste, altrimenti
16         // restituisce m.end().
17         auto it = m.find(chiave);
18
19         if (it != m.end()) {
20             // (*it).first      la chiave, (*it).second      il
21             // valore
22             cout << "Trovato: " << it->first << " -> " << it->
23             second << endl;
24         } else {
25             cout << "Chiave non trovata." << endl;
26         }
27
28         return 0;
29     }

```

Complessità: $O(\log n)$

Nota: `(*it.first` e `it->first`) sono la stessa cosa.

- **Controllo se vuota:** `m.empty()` Restituisce `true` se la map non contiene elementi.

2.2.3 Altre operazioni utili

- `m.count(key)` → restituisce 1 se la chiave esiste, 0 altrimenti.

2.2.4 Iterazione su una map

```

1      for (auto it = m.begin(); it != m.end(); ++it) {
2          cout << it->first << " -> " << it->second << endl;
3      }
4
5      // range-based for (C++11+)
6      for (auto [key, value] : m) {
7          cout << key << " -> " << value << endl;
8      }

```

Note:

- `it->first` accede alla chiave, `it->second` al valore.
- Gli elementi sono ordinati automaticamente in base alla chiave (in ordine crescente di default, cioè secondo l'operatore `<` della chiave).

2.2.5 Complessità computazionale riassunta

Operazione	Complessità
Accesso <code>m[i]</code>	$O(\log n)$
Ricerca <code>m.find(key)</code>	$O(\log n)$
Inserimento <code>m.insert(key, value)</code>	$O(\log n)$
Cancellazione <code>m.erase(key)</code>	$O(\log n)$
Dimensione <code>m.size()</code>	$O(1)$
Controllo se vuota <code>m.empty()</code>	$O(1)$

Tabella 2.2: Complessità operazioni principali sulle map

2.3 Set

Il **set** è una struttura dati della Standard Template Library (STL) che rappresenta un **contenitore associativo ordinato** di elementi **unici**. Ogni elemento è sia la chiave che il valore, e viene automaticamente ordinato in base al suo valore.

In figura 2.7 si può notare come gli elementi siano ordinati in modo crescente e non siano presenti duplicati.

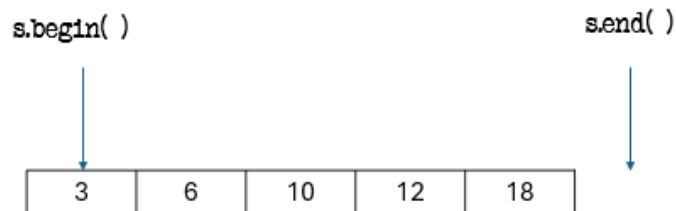


Figura 2.7: Rappresentazione grafica di un **set**

2.3.1 Dichiarazione di un set

```
1  #include <set>
2  #include <iostream>
3  using namespace std;
4
5  set<int> s1;           // set vuoto di interi
6
7  set<int> s2 = {5, 1, 9, 1, 3}; // inizializzazione con duplicati
8  // Risultato: {1, 3, 5, 9} (gli elementi duplicati vengono
9  // ignorati)
10
11 set<string, greater<string>> s3 = {"mela", "pera", "banana"};
// greater<string> ordina in modo decrescente
```

Note:

- Gli elementi di un set sono sempre univoci.
- La struttura mantiene automaticamente l'ordine.
- Se si vuole un set non ordinato, si può usare `unordered_set`.

2.3.2 Operazioni principali sui set

Le operazioni che seguono fanno riferimento al set della figura 2.7.

- **Inserimento di un elemento:** `s.insert(valore)`

```
1  s.insert(4);           // inserisce l'elemento 4
2  s.insert(3);           // ignorato se 3 e' gia' presente
```

Complessità: $O(\log n)$

- **Rimozione di un elemento:** `s.erase(valore)`

```
1 s.erase(1); // rimuove l'elemento 1 se presente
```

Complessità: $O(\log n)$

- **Ricerca di un elemento:** `s.find(valore)`

```
1 auto it = s.find(3);
2 if (it != s.end())
3     cout << "Trovato: " << *it;
4 else
5     cout << "Elemento non trovato";
```

Complessità: $O(\log n)$

- **Verifica esistenza di un elemento:** `s.count(valore)`

```
1 if (s.count(5)) cout << "Presente!";
2 else cout << "Assente!";
```

Restituisce 1 se l'elemento esiste, 0 altrimenti. Complessità: $O(\log n)$

- **Dimensione:** `s.size()`

```
1 cout << s.size(); // numero di elementi nel set
```

Complessità: $O(1)$

- **Controllo se vuoto:** `s.empty()`

```
1 if (s.empty()) cout << "Set vuoto";
```

Complessità: $O(1)$

Accesso ad un elemento:

L'accesso ad un elemento di un **set** avviene mediante dereferenziazione dell'iteratore.

Questa volta però, a differenza dei **vector**, non possiamo accedere ad un elemento sommando un intero all'iteratore:

```
1 #include <iostream>
2 #include <set>
3 using namespace std;
4
5 int main() {
6     set<int> s = {1, 3, 5, 7};
7
8     it = s.begin();
9
10    // voglio stampare il 4 elemento
11    cout << s.begin() + 3; // stampa un errore!
12
13    return 0;
14 }
```

L'errore è dovuto al fatto che gli iteratori dei **set** non supportano l'aritmetica dei puntatori. Possiamo solo avanzare o retrocedere di una posizione alla volta (con `it++` e `it--`).

Quindi se volessi accedere al terzo elemento, una soluzione potrebbe essere:

```

1      #include <iostream>
2      #include <set>
3      using namespace std;
4
5      int main() {
6          set<int> s = {1, 3, 5, 7};
7
8          it = s.begin();
9
10         it++;
11         it++;
12
13         // Attenzione: fare it = it + 2      errato in quanto gli
            iteratori dei set non supportano l'aritmetica dei
            puntatori!
14
15         cout << *it; // Stampa 5
16
17         return 0;
18     }

```

Un'alternativa è utilizzare la libreria `<iterator>`, che mette a disposizione la funzione **`advance(iteratore, n)`**. Questa funzione permette di spostare l'iteratore di `n` posizioni lungo il contenitore.

È doveroso notare che la funzione `advance(valore, n)` può andare ben oltre la posizione `s.end()`. Ciò significa che è opportuno verificare che il set abbia elementi sufficienti per accedere alla posizione desiderata:

```

1      #include <iostream>
2      #include <set>
3      #include <iterator>
4      using namespace std;
5
6      int main() {
7          set<int> s = {1, 3, 5, 7, 9};
8
9          // controlliamo prima che il set abbia almeno 3
            elementi
10         if (s.size() >= 3) {
11             auto it = s.begin();          // iteratore all'inizio
12             advance(it, 2);                // spostiamo l'iteratore
            di 2 posizioni (3 elemento)
13             cout << "Il terzo elemento e': " << *it << endl;
14         } else {
15             cout << "Il set ha meno di 3 elementi." << endl;
16         }
17
18         return 0;
19     }

```

2.3.3 Altre operazioni utili

- **lower_bound**

Restituisce un iteratore al primo elemento **maggiore o uguale** al valore passato come parametro. Restituisce `s.end()` se non trova nulla.

```
1      #include <iostream>
2      #include <set>
3      using namespace std;
4
5      int main() {
6          set<int> s = {1, 3, 5, 7};
7
8          auto it = s.lower_bound(3); // primo elemento >= 3
9          if(it != s.end())
10             cout << "lower_bound(3) = " << *it << endl;
11
12             return 0;
13     }
```

Output: lower_bound(3) = 3

- **upper_bound**

Restituisce un iteratore al primo elemento **maggiore** del valore passato come parametro. `s.end()` se non trova nulla.

```
1      #include <iostream>
2      #include <set>
3      using namespace std;
4
5      int main() {
6          set<int> s = {1, 3, 5, 7};
7
8          auto it = s.upper_bound(3); // primo elemento > 3
9          if(it != s.end())
10             cout << "upper_bound(3) = " << *it << endl;
11
12             return 0;
13     }
```

Output: upper_bound(3) = 5

2.3.4 Iterazione su un set

```
1   for (auto it = s.begin(); it != s.end(); ++it)
2       cout << *it << " ";
3
4   // range-based for (C++11+)
5   for (auto x : s)
6       cout << x << " ";
```

Note:

- L'iteratore restituisce il valore direttamente, non una coppia chiave-valore.
- Gli elementi sono ordinati automaticamente.

2.3.5 Esempio pratico

```
1   #include <set>
2   #include <iostream>
3   using namespace std;
4
5   int main() {
6       set<int> s;
7
8       s.insert(10);
9       s.insert(5);
10      s.insert(10);
11      s.insert(8);
12
13      cout << "Elementi nel set: ";
14      for (int x : s) cout << x << " ";
15
16      cout << "\nDimensione: " << s.size() << endl;
17
18      if (s.find(5) != s.end())
19          cout << "5 e' presente!\n";
20
21      s.erase(8);
22      cout << "Dopo cancellazione: ";
23      for (int x : s) cout << x << " ";
24
25      return 0;
26  }
```

Output:

```
Elementi nel set: 5 8 10
Dimensione: 3
5 e' presente!
Dopo cancellazione: 5 10
```

2.3.6 Complessità computazionale riassunta

In tabella 2.3 sono state riassunte le complessità computazionali delle operazioni sui **set**³

Operazione	Complessità
Inserimento <code>s.insert(x)</code>	$O(\log n)$
Ricerca <code>s.find(x)</code>	$O(\log n)$
Cancellazione <code>s.erase(x)</code>	$O(\log n)$
Controllo esistenza <code>s.count(x)</code>	$O(\log n)$
lower_bound <code>s.lower_bound(x)</code>	$O(\log n)$
upper_bound <code>s.upper_bound(x)</code>	$O(\log n)$
Dimensione <code>s.size()</code>	$O(1)$
Controllo se vuoto <code>s.empty()</code>	$O(1)$

Tabella 2.3: Complessità operazioni principali sui set

³La funzione `advance(it, n)` ha complessità $O(n)$

2.4 Multiset

Il **multiset** è una struttura dati della libreria standard di C++ molto simile al **set**, con una differenza fondamentale: mentre nel **set** ogni elemento è unico, nel **multiset** possono esistere più copie dello stesso valore.

```
1  #include <iostream>
2  #include <set>
3  using namespace std;
4
5  int main() {
6      multiset<int> ms;
7      ms.insert(5);
8      ms.insert(5);
9      ms.insert(3);
10     ms.insert(7);
11
12     for (int x : ms)
13         cout << x << " "; // stampa: 3 5 5 7
14 }
```

Come il **set**, anche il **multiset** memorizza gli elementi in modo ordinato e permette di eseguire operazioni di inserimento, ricerca e cancellazione in $O(\log n)$.

2.4.1 Caratteristiche principali

- Gli elementi sono **ordinati automaticamente**.
- È possibile inserire **valori duplicati**.
- Non è possibile accedere agli elementi tramite indice.
- Gli iteratori sono bidirezionali, quindi non supportano operazioni aritmetiche ($it + n$).

2.4.2 Esempio pratico

```
1  #include <iostream>
2  #include <set>
3  using namespace std;
4
5  int main() {
6      multiset<int> ms = {2, 3, 3, 5, 5, 5};
7
8      cout << "Numero di 5: " << ms.count(5) << endl; // 3
9
10     ms.erase(ms.find(3)); // cancella solo una delle due occorrenze
11                             di 3
12
13     for (int x : ms)
14         cout << x << " "; // stampa: 2 3 5 5 5
15 }
```

2.4.3 Complessità computazionale riassunta

Operazione	Complessità
Inserimento <code>ms.insert(x)</code>	$O(\log n)$
Ricerca <code>ms.find(x)</code>	$O(\log n)$
Cancellazione di un elemento <code>ms.erase(it)</code>	$O(1)$
Cancellazione di tutti gli elementi con un certo valore <code>ms.erase(x)</code>	$O(\log n + k)^4$
Conteggio occorrenze <code>ms.count(x)</code>	$O(\log n + k)$
<code>lower_bound</code> <code>ms.lower_bound(x)</code>	$O(\log n)$
<code>upper_bound</code> <code>ms.upper_bound(x)</code>	$O(\log n)$
Dimensione <code>ms.size()</code>	$O(1)$
Controllo se vuoto <code>ms.empty()</code>	$O(1)$

Tabella 2.4: Complessità operazioni principali sui multiset

⁴Nella tabella delle complessità del multiset, la lettera **k** indica il numero di elementi con lo stesso valore (cioè il numero di duplicati). Infatti `ms.erase(x)` deve cancellare **k** occorrenze e, di conseguenza, la complessità risulta $O(\log n)$ per la ricerca più **k** per eliminare ogni occorrenza)

2.5 Stack

Lo **stack** (in italiano: pila) è una struttura dati della Standard Template Library (STL) che segue il principio **LIFO** (*Last In, First Out*), ovvero l'ultimo elemento inserito è il primo a essere rimosso.

In figura 2.8 è mostrata la logica di funzionamento di una pila: gli elementi vengono aggiunti (push) e rimossi (pop) sempre dalla stessa estremità.

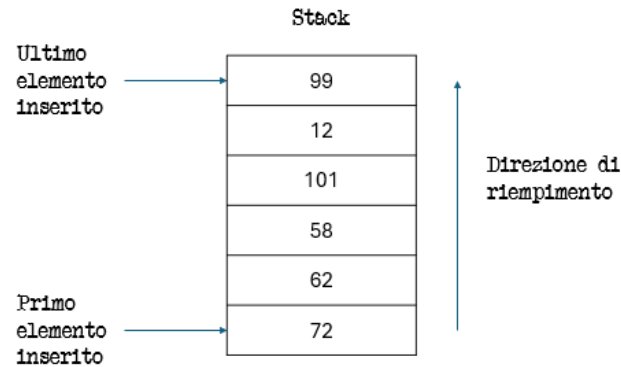


Figura 2.8: Rappresentazione grafica di uno **stack**

2.5.1 Dichiarazione di uno stack

```
1  #include <stack>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      stack<int> s;      // dichiara uno stack di interi vuoto
7
8      s.push(10);        // inserisce 10
9      s.push(20);        // inserisce 20
10     s.push(30);        // inserisce 30
11
12     cout << "Top: " << s.top() << endl; // stampa 30
13
14     s.pop();            // rimuove l'ultimo elemento inserito (30)
15
16     cout << "Top dopo pop: " << s.top() << endl; // stampa 20
17 }
```

Note:

- Lo stack non consente accesso diretto agli elementi (come con gli indici).
- È possibile accedere solo all'elemento in cima (`top()`).
- La classe **stack** è un *adattatore di contenitore*, che di default usa un **deque** come contenitore sottostante.

2.5.2 Operazioni principali sullo stack

Le operazioni principali disponibili sono:

- **push(valore)** – Inserisce un nuovo elemento in cima alla pila.

```
1 s.push(42);
```

Complessità: $O(1)$

- **pop()** – Rimuove l'elemento in cima alla pila.

```
1 s.pop();
```

Complessità: $O(1)$

- **top()** – Restituisce una **riferimento** all'elemento in cima.

```
1 cout << s.top();
```

Complessità: $O(1)$

- **empty()** – Verifica se la pila è vuota.

```
1 if (s.empty()) cout << "Stack vuoto!";
```

Complessità: $O(1)$

- **size()** – Restituisce il numero di elementi presenti nello stack.

```
1 cout << s.size();
```

Complessità: $O(1)$

2.5.3 Iterazione su uno stack

Lo **stack non supporta l'iterazione diretta** (come con `for(auto x : s)` o iteratori), poiché è pensato per un accesso controllato di tipo LIFO. Tuttavia, è possibile svuotarlo progressivamente per leggere tutti i suoi elementi:

```
1 #include <stack>
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     stack<int> s;
7     s.push(10);
8     s.push(20);
9     s.push(30);
10
11     while (!s.empty()) {
12         cout << s.top() << " ";
13         s.pop();
14     }
15     // Output: 30 20 10
16 }
```

Nota: Questo metodo svuota lo stack. Se serve mantenerlo, conviene copiarlo prima in un altro stack.

2.5.4 Esempio pratico

```
1  #include <stack>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      stack<string> history;
7
8      history.push("www.google.com");
9      history.push("www.openai.com");
10     history.push("www.github.com");
11
12     cout << "Pagina corrente: " << history.top() << endl;
13     history.pop();
14
15     cout << "Torno a: " << history.top() << endl;
16
17     cout << "Totale pagine: " << history.size() << endl;
18 }
```

Output:

Pagina corrente: www.github.com
Torno a: www.openai.com
Totale pagine: 2

2.5.5 Complessità computazionale riassunta

Operazione	Complessità
Inserimento <code>s.push(x)</code>	$O(1)$
Rimozione <code>s.pop()</code>	$O(1)$
Accesso al top <code>s.top()</code>	$O(1)$
Controllo se vuoto <code>s.empty()</code>	$O(1)$
Dimensione <code>s.size()</code>	$O(1)$

Tabella 2.5: Complessità operazioni principali sullo stack

2.6 Queue

La **queue** (in italiano: coda) è una struttura dati della STL che segue il principio **FIFO** (*First In, First Out*), ovvero il primo elemento inserito è il primo a essere rimosso. Si comporta come una vera e propria coda di persone: chi entra per primo esce per primo.

Di fatto è il contrario di uno **stack**

In figura 2.9 è illustrato il funzionamento logico della coda.

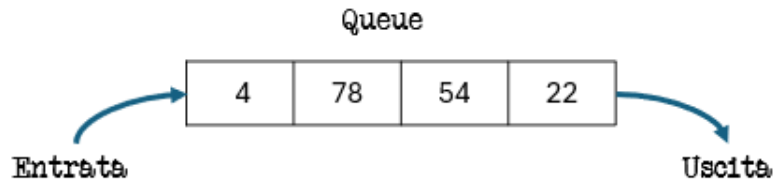


Figura 2.9: Rappresentazione grafica di una **queue**

2.6.1 Dichiarazione di una queue

```
1  #include <queue>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      queue<int> q;          // dichiara una coda di interi
7
8      q.push(10);            // inserisce 10 in fondo alla coda
9      q.push(20);            // inserisce 20
10     q.push(30);            // inserisce 30
11
12     cout << "Front: " << q.front() << endl; // stampa 10
13     cout << "Back: " << q.back() << endl;  // stampa 30
14
15     q.pop();                // rimuove il primo elemento (10)
16
17     cout << "Front dopo pop: " << q.front() << endl; // stampa 20
18 }
```

Note:

- È possibile accedere solo all'elemento **front** (testa) e a quello **back** (coda).
- Non è possibile accedere direttamente agli elementi centrali o iterare sulla coda.
- Internamente, la **queue** è un **adattatore di contenitore**, che usa di default un deque.

2.6.2 Operazioni principali sulla queue

- **push(valore)** – Inserisce un elemento in fondo alla coda.

```
1  q.push(42);
```

Complessità: $O(1)$

- **pop()** – Rimuove l'elemento in testa alla coda.

```
1 q.pop();
```

Complessità: $O(1)$

- **front()** – Restituisce una **riferimento** all'elemento in testa.

```
1 cout << q.front();
```

Complessità: $O(1)$

- **back()** – Restituisce una **riferimento** all'ultimo elemento inserito.

```
1 cout << q.back();
```

Complessità: $O(1)$

- **empty()** – Verifica se la coda è vuota.

```
1 if (q.empty()) cout << "Coda vuota!";
```

Complessità: $O(1)$

- **size()** – Restituisce il numero di elementi nella coda.

```
1 cout << q.size();
```

Complessità: $O(1)$

2.6.3 Iterazione su una queue

Come lo **stack**, anche la **queue** non supporta iteratori né accesso diretto. È possibile tuttavia svuotarla per leggere i valori in ordine di inserimento:

```
1  #include <queue>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      queue<int> q;
7      q.push(10);
8      q.push(20);
9      q.push(30);
10
11     while (!q.empty()) {
12         cout << q.front() << " ";
13         q.pop();
14     }
15     // Output: 10 20 30
16 }
```

Nota: Questo metodo svuota la coda. Se serve mantenere i dati, conviene copiarla prima in un'altra queue.

2.6.4 Esempio pratico

```
1  #include <queue>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      queue<string> clienti;
7
8      clienti.push("Marco");
9      clienti.push("Giulia");
10     clienti.push("Antonio");
11
12     cout << "Serve: " << clienti.front() << endl;
13     clienti.pop();
14
15     cout << "Prossimo cliente: " << clienti.front() << endl;
16     cout << "Ultimo in coda: " << clienti.back() << endl;
17 }
```

Output:

Serve: Marco
Prossimo cliente: Giulia
Ultimo in coda: Antonio

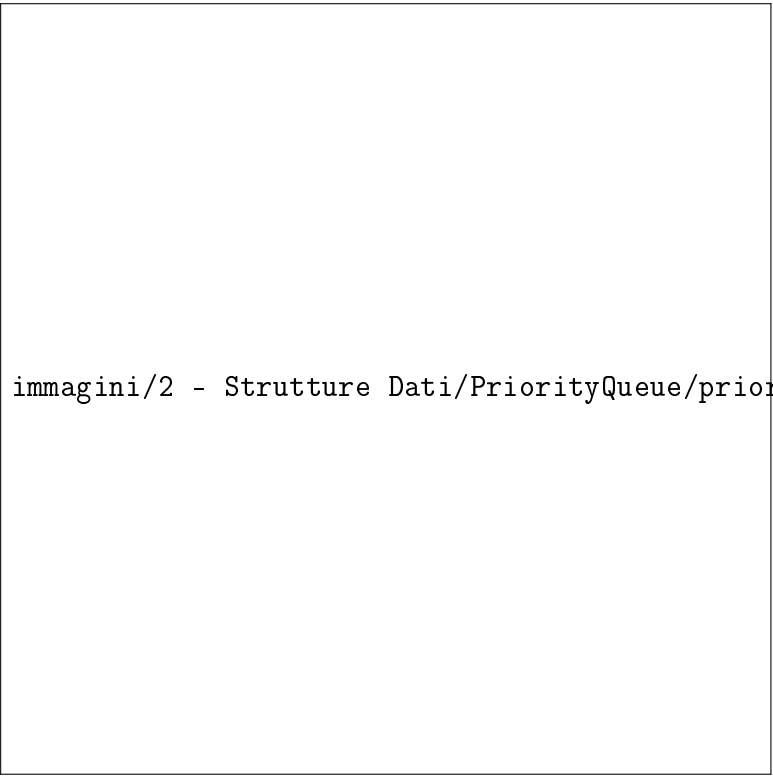
2.6.5 Complessità computazionale riassunta

Operazione	Complessità
Inserimento in coda <code>q.push(x)</code>	$O(1)$
Rimozione in testa <code>q.pop()</code>	$O(1)$
Accesso al primo elemento <code>q.front()</code>	$O(1)$
Accesso all'ultimo elemento <code>q.back()</code>	$O(1)$
Controllo se vuota <code>q.empty()</code>	$O(1)$
Dimensione <code>q.size()</code>	$O(1)$

Tabella 2.6: Complessità operazioni principali sulla queue

2.7 Priority Queue

La **priority_queue** è una struttura dati della STL che memorizza gli elementi secondo una **priorità**. A differenza della **queue** tradizionale, non segue l'ordine FIFO, ma estrae sempre l'elemento con la priorità più alta (di default il valore massimo).



immagini/2 - Strutture Dati/PriorityQueue/priority_queue.png

Figura 2.10: Rappresentazione grafica di una **priority_queue**

2.7.1 Dichiarazione di una **priority_queue**

```
1  #include <queue>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      priority_queue<int> pq; // priority_queue di interi (max-heap)
7
8      pq.push(10);
9      pq.push(5);
10     pq.push(20);
11
12     cout << "Elemento con priorit massima: " << pq.top() << endl;
13     // 20
14
15     priority_queue<string> pq2;
16
17     pq2.push("banana");
18     pq2.push("mela");
19     pq2.push("arancia");
```

```

20     cout << "Massimo: " << pq2.top() << endl;
21     // stampa mela in quanto mela considerata massima.
        Confrontando le stringhe alfabeticamente, viene dopo banana
        e arancia.
22
23 }

```

Note:

- Gli elementi sono ordinati automaticamente in base alla priorità.
- Di default è una **max-heap** (massimo elemento in cima).
- Per creare una **min-heap** si usa:

```

1     priority_queue<int, vector<int>, greater<int>> pq_min;

```

2.7.2 Operazioni principali sulla priority_queue

- **push(valore)** – Inserisce un elemento nella coda di priorità.

```

1     pq.push(42);

```

Complessità: $O(\log n)$

- **pop()** – Rimuove l'elemento con priorità più alta.

```

1     pq.pop();

```

Complessità: $O(\log n)$

- **top()** – Restituisce un riferimento all'elemento con priorità più alta.

```

1     cout << pq.top();

```

Complessità: $O(1)$

- **empty()** – Verifica se la coda è vuota.

```

1     if (pq.empty()) cout << "Priority queue vuota!";

```

Complessità: $O(1)$

- **size()** – Restituisce il numero di elementi presenti.

```

1     cout << pq.size();

```

Complessità: $O(1)$

2.7.3 Iterazione su una priority_queue

Così come la queue, non è possibile iterare direttamente o accedere agli elementi centrali.

2.7.4 Esempio pratico

```
1  #include <queue>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      priority_queue<int> pq;
7
8      pq.push(15);
9      pq.push(30);
10     pq.push(10);
11
12     cout << "Massimo elemento: " << pq.top() << endl; // 30
13     pq.pop();
14
15     cout << "Nuovo massimo: " << pq.top() << endl; // 15
16     cout << "Dimensione: " << pq.size() << endl; // 2
17 }
```

Output:

```
Massimo elemento: 30
Nuovo massimo: 15
Dimensione: 2
```

2.7.5 Complessità computazionale riassunta

Operazione	Complessità
Inserimento <code>pq.push(x)</code>	$O(\log n)$
Rimozione del massimo <code>pq.pop()</code>	$O(\log n)$
Accesso al massimo <code>pq.top()</code>	$O(1)$
Controllo se vuota <code>pq.empty()</code>	$O(1)$
Dimensione <code>pq.size()</code>	$O(1)$

Tabella 2.7: Complessità operazioni principali sulla `priority_queue`

Nota: la `priority_queue` è molto efficiente quando in un insieme di dati dobbiamo accedere frequentemente all'elemento massimo. Se invece è necessario poter iterare sulla struttura o accedere ad elementi “centrali”, è più conveniente utilizzare un `multiset`.

2.8 Deque

La **deque** (double-ended queue, coda a doppia estremità) è una struttura dati della STL che permette l'inserimento e la rimozione di elementi sia all'inizio sia alla fine del contenitore. Si comporta come una combinazione tra **vector** e **queue**: supporta accesso casuale agli elementi e operazioni efficienti alle estremità.

In figura 2.11 è illustrato il funzionamento logico di una **deque**.

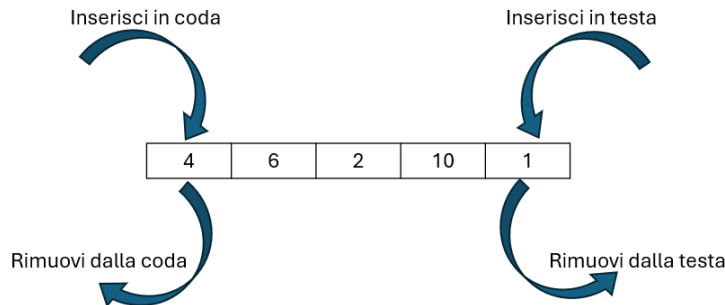


Figura 2.11: Rappresentazione grafica di una **deque**

2.8.1 Dichiarazione di una deque

```
1  #include <deque>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      deque<int> d;           // deque vuota di interi
7
8      d.push_back(10);        // inserisce 10 in coda
9      d.push_front(5);        // inserisce 5 in testa
10     d.push_back(20);         // inserisce 20 in coda
11
12     cout << "Front: " << d.front() << endl; // stampa 5
13     cout << "Back: " << d.back() << endl;   // stampa 20
14 }
```

Note:

- La **deque** permette inserimenti e cancellazioni sia in testa sia in coda in tempo costante $O(1)$.
- Supporta accesso casuale agli elementi tramite l'operatore `[]`, come un **vector**.
- È più flessibile di un **queue** o **stack** perché consente operazioni su entrambe le estremità.

2.8.2 Operazioni principali sulla deque

- **push_front(valore)** – Inserisce un elemento all'inizio.

```
1  d.push_front(3);
```

Complessità: $O(1)$

- **push_back(valore)** – Inserisce un elemento alla fine.

```
1 d.push_back(7);
```

Complessità: $O(1)$

- **pop_front()** – Rimuove l'elemento in testa.

```
1 d.pop_front();
```

Complessità: $O(1)$

- **pop_back()** – Rimuove l'elemento in coda.

```
1 d.pop_back();
```

Complessità: $O(1)$

- **front()** – Restituisce riferimento al primo elemento.

```
1 cout << d.front();
```

Complessità: $O(1)$

- **back()** – Restituisce riferimento all'ultimo elemento.

```
1 cout << d.back();
```

Complessità: $O(1)$

- **operator[]** – Accesso casuale ad un elemento.

```
1 cout << d[1]; // secondo elemento
```

Complessità: $O(1)$

- **size()** – Numero di elementi.

```
1 cout << d.size();
```

Complessità: $O(1)$

- **empty()** – Verifica se la deque è vuota.

```
1 if(d.empty()) cout << "Deque vuota!";
```

Complessità: $O(1)$

2.8.3 Iterazione su una deque

```
1 for(auto it = d.begin(); it != d.end(); ++it)
2   cout << *it << " ";
3
4 // oppure con range-based for
5 for(int x : d)
6   cout << x << " ";
```

2.8.4 Esempio pratico

```
1  #include <deque>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      deque<int> d;
7
8      d.push_back(10);
9      d.push_front(5);
10     d.push_back(20);
11
12     cout << "Elementi nella deque: ";
13     for(int x : d) cout << x << " ";
14
15     d.pop_front();
16     cout << "\nDopo pop_front: ";
17     for(int x : d) cout << x << " ";
18
19     d.pop_back();
20     cout << "\nDopo pop_back: ";
21     for(int x : d) cout << x << " ";
22
23     return 0;
24 }
```

Output:

```
Elementi nella deque: 5 10 20
Dopo pop_front: 10 20
Dopo pop_back: 10
```

2.8.5 Complessità computazionale riassunta

Operazione	Complessità
Inserimento in testa <code>push_front(x)</code>	$O(1)$
Inserimento in coda <code>push_back(x)</code>	$O(1)$
Rimozione in testa <code>pop_front()</code>	$O(1)$
Rimozione in coda <code>pop_back()</code>	$O(1)$
Accesso al primo elemento <code>front()</code>	$O(1)$
Accesso all'ultimo elemento <code>back()</code>	$O(1)$
Accesso casuale <code>operator[]</code>	$O(1)$
Controllo se vuota <code>empty()</code>	$O(1)$
Dimensione <code>size()</code>	$O(1)$

Tabella 2.8: Complessità operazioni principali sulla deque

2.8.6 Differenze tra queue e deque

A questo punto potremmo chiederci perché usare una **queue** quando abbiamo a disposizione la **deque** che, effettivamente, fa più cose senza aumentare la complessità computazionale.

La differenza principale tra **queue** e **deque**, aldilà delle funzionalità base, sta nel tipo di astrazione che vogliamo utilizzare.

- **queue**
 - È un **adattatore di contenitore**: ci interessa solo il comportamento FIFO.
 - Non permette l'accesso agli elementi centrali né l'iterazione.
 - L'interfaccia è semplice: `push`, `pop`, `front`, `back`.
 - Vantaggio: garantisce **chiarezza e sicurezza**, perché chi legge il codice sa subito che si sta usando una coda "pura".
- **deque**
 - È un **contenitore generico**: permette quasi tutte le operazioni, come iterazione, accesso casuale, inserimenti/rimozioni sia in testa che in coda.
 - Offre più flessibilità, ma anche più possibilità di usare la struttura in modo non coerente con una coda FIFO.

In pratica:

- Se vogliamo solo una coda, usiamo **queue**: comunica chiaramente l'intento e riduce il rischio di errori.

- Se vogliamo più flessibilità (accesso casuale, iterazione, manipolazioni complesse), usiamo deque.

È come scegliere tra un coltellino e un coltellino svizzero: il coltellino fa una cosa e la fa bene, il coltellino può fare di più ma non sempre è la scelta più chiara.

Capitolo 3

Brute Force e Algoritmi Greedy

In questo capitolo analizzeremo due approcci fondamentali alla risoluzione dei problemi: gli algoritmi **Brute Force** e gli algoritmi **Greedy**. Si tratta di due paradigmi concettualmente semplici, ma molto diversi tra loro per strategia, prestazioni e casi d'uso.

Il **Brute Force** adotta un approccio esaustivo: esplora tutte le soluzioni possibili e seleziona quella corretta o quella ottimale. È un metodo semplice, intuitivo e sempre corretto, ma spesso troppo lento per input di dimensioni elevate.

Gli algoritmi **Greedy**, al contrario, seguono una strategia “miope”: a ogni passo scelgono la soluzione che sembra migliore in quel momento, senza considerare le conseguenze future. Questo permette grande efficienza, ma non garantisce sempre una soluzione ottimale.

3.1 Algoritmi Brute Force

Gli algoritmi Brute Force rappresentano il metodo più semplice e diretto per risolvere un problema: consistono nel generare e valutare **tutte** le soluzioni possibili, scegliendo poi quella corretta o quella ottimale. Questo approccio è intuitivo e non richiede strutture dati o strategie sofisticate: si limita a esplorare in maniera esaustiva l'intero spazio delle soluzioni.

La forza del metodo Brute Force risiede nella sua semplicità: poiché vengono considerate tutte le possibilità, l'algoritmo garantisce sempre la correttezza della soluzione trovata. Tuttavia, la sua debolezza è altrettanto evidente: il numero di combinazioni cresce spesso in modo **esponenziale**, rendendo il Brute Force impraticabile per problemi di dimensioni medio-grandi.

Nonostante ciò, gli algoritmi Brute Force sono fondamentali per diversi motivi:

- sono facili da implementare e utili come punto di partenza;
- forniscono una soluzione ottima, quando è possibile enumerare tutto lo spazio delle soluzioni;
- permettono di confrontare la qualità di algoritmi più avanzati;
- sono spesso accettabili quando i dati in input sono piccoli oppure quando il problema non richiede una soluzione immediata.

3.1.1 Antivirus

Link al problema

Time limit = 1.00s, Memory limit = 512 MB

Il nuovo sistema di gara delle Selezioni Territoriali funziona alla grande, ma sembra che la mascotte abbia fiutato un virus nascosto fra i file inviati da un partecipante! Conosciamo la lunghezza del virus e sappiamo che si ripete uguale nei quattro file ricevuti, ma non sappiamo dove. Aiutaci a individuare il virus!

Dettagli:

Sono dati in input quattro file F_1, F_2, F_3, F_4 , rappresentati come quattro stringhe di caratteri di lunghezza rispettivamente N_1, N_2, N_3, N_4 .

Il virus è una stringa V di lunghezza M . La lunghezza M è nota (viene fornita in input), ma non conosciamo il contenuto della stringa V del virus. Sappiamo con certezza che V appare all'interno di tutti e quattro i file, come sottostringa di caratteri *consecutivi*. Sappiamo inoltre che *NON* ci sono altre sottostringhe consecutive di lunghezza M che si ripetono uguali in tutti e quattro i file.

Le posizioni dei caratteri nelle stringhe sono numerati a partire da 0. Per ciascuno dei quattro file F_i , trova la posizione in cui è inserito il virus, ovvero la posizione dove appare il primo carattere del virus V all'interno della stringa F_i .

L'obiettivo è: per ciascuno dei quattro file F_i , trovare la posizione (indice, a partire da 0) in cui compare V , ovvero l'indice del primo carattere di V dentro F_i .

Formato di input

La prima riga contiene un intero T , il numero di casi di test. Seguono T casi di test, ognuno preceduto da una riga vuota.

Per ciascun caso di test:

- Una riga con quattro interi N_1, N_2, N_3, N_4 — le lunghezze dei quattro file.
- Una riga con un intero M — la lunghezza del virus.
- Quattro righe contenenti le stringhe F_1, F_2, F_3, F_4 .

Formato di output

Per ogni caso di test, stampare:

Case #t: $p_1 p_2 p_3 p_4$

dove t è il numero del caso (a partire da 1), e p_i è la posizione (indice 0-based) della prima occorrenza della stringa virus V dentro F_i .

Assunzioni e Vincoli

- $T = 12$
- $2 \leq N_1, N_2, N_3, N_4 \leq 100$
- $2 \leq M \leq 20$
- $M \leq \min(N_1, N_2, N_3, N_4)$
- Tutti i caratteri dei file sono lettere minuscole dell'alfabeto inflese (dalla a alla z), NON sono presenti spazi.

- È garantito che il virus esiste ed è unico

Esempio

Input:

```

2

8 12 10 7
4
ananasso
associazione
tassonomia
massone

6 9 11 10
3
simone
ponessimo
milionesimo
cassonetto

```

Output:

```

Case #1: 4 0 1 1
Case #2: 3 1 4 4

```

Spiegazione:

Nel primo caso il virus è “asso”: **an**an**asso**, **ass**ociazione, **tass**onomia, **mass**one.

Nel secondo caso il virus è “one”: **sim**one, **pon**essimo, **milione**simo, **cassone**tto.

Idea (brute force)

Si considerano tutte le possibili sottostringhe di lunghezza M della prima stringa F_1 . Ogni sottostringa di questo tipo costituisce una possibile candidata a essere il virus.

Per ciascuna sottostringa candidata:

- si verifica se essa compare almeno una volta nella stringa F_2 ;
- si verifica se essa compare almeno una volta nella stringa F_3 ;
- si verifica se essa compare almeno una volta nella stringa F_4 .

Il controllo viene effettuato confrontando la sottostringa candidata con tutte le sottostringhe di lunghezza M delle altre stringhe.

Appena una sottostringa candidata risulta presente in tutte e quattro le stringhe, essa è necessariamente il virus, poiché il problema ne garantisce l'esistenza e l'unicità; a questo punto l'algoritmo può terminare.

Infine, si memorizzano le posizioni in cui il virus compare all'interno di ciascuna stringa.

Implementazione C++ - Brute Force

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      // Apro il file di input
7      ifstream inputFile("antivirus_input_1.txt");
8
9      int T;                                // Numero di test case nel file
10     inputFile >> T;
11
12     // Ciclo su tutti i test case
13     for (int test = 1; test <= T; ++test)
14     {
15         // N1, N2, N3, N4 sono le lunghezze delle quattro stringhe
16         // F1, F2, F3, F4
17         int N1, N2, N3, N4;
18         inputFile >> N1 >> N2 >> N3 >> N4;
19
20         // M la lunghezza della sottostringa del virus
21         int M;
22         inputFile >> M;
23
24         // Le quattro stringhe in cui cercare il virus
25         string F1, F2, F3, F4;
26         inputFile >> F1 >> F2 >> F3 >> F4;
27
28         // Variabili per salvare la posizione in cui il virus
29         // appare in ciascuna stringa
30         int F1_virus; // Posizione in F1 da cui inizia la
31         // sottostringa candidata
32         int F2_virus; // Posizione in F2 dove compare la
33         // sottostringa candidata
34         int F3_virus; // Posizione in F3 dove compare la
35         // sottostringa candidata
36         int F4_virus; // Posizione in F4 dove compare la
37         // sottostringa candidata
38
39         /*
40         Ciclo principale: scorriamo tutte le sottostringhe di
41         lunghezza M in F1
42         e le consideriamo candidate al virus
43         */
44         for(int i = 0; i <= F1.length(); i++){
45             F1_virus = i; // La sottostringa candidata parte da
46             // posizione i in F1
47             F2_virus = -1; // Inizializzo a -1, se rimane -1 vuol
48             // dire che non l'ho trovata
49             F3_virus = -1;
50             F4_virus = -1;
```

```

43         // Estraggo la sottostringa candidata dalla stringa F1
44         string virus_candidato = F1.substr(i, M);
45
46         // Cerco il virus in F2
47         for(int j = 0; j < F2.length(); j++){
48             if(F2.substr(j, M) == virus_candidato){ //
49                 confronto ogni sottostringa di F2
50                 F2_virus = j; // trovato, salvo la posizione
51                 break; // esco dal ciclo
52             }
53         }
54
55         // Cerco il virus in F3
56         for(int j = 0; j < F3.length(); j++){
57             if(F3.substr(j, M) == virus_candidato){
58                 F3_virus = j;
59                 break;
60             }
61         }
62
63         // Cerco il virus in F4
64         for(int j = 0; j < F4.length(); j++){
65             if(F4.substr(j, M) == virus_candidato){
66                 F4_virus = j;
67                 break;
68             }
69         }
70
71         // Se la sottostringa candidata e' presente in tutte e
72         // quattro le stringhe, abbiamo trovato il virus
73         if(F2_virus != -1 && F3_virus != -1 && F4_virus != -1){
74             break; // esco dal ciclo su F1
75         }
76
77         // Stampo le posizioni trovate per ciascuna stringa
78         cout << "Case #" << test << ": "
79         << F1_virus << " "
80         << F2_virus << " "
81         << F3_virus << " "
82         << F4_virus << endl;
83     }
84
85     inputFile.close(); // Chiudo il file di input
86     return 0;
87 }

```

Costo Computazionale

Il costo computazionale è:

- $O(N_2) + O(N_3) + O(N_4)$ per i cicli interni nelle righe 47, 55, 63;
- $O(N_1)$ per il ciclo esterno alla riga 37.

Ciò significa che i cicli più interni vengono eseguiti N_1 volte.
La complessità totale è quindi:

$$\begin{aligned} &= O(N_1) \cdot [O(N_2) + O(N_3) + O(N_4)] \\ &\approx O(N_1) \cdot \max(O(N_2), O(N_3), O(N_4)) \end{aligned}$$

Dai vincoli del problema:

$$2 \leq N_i \leq 100$$

la complessità può essere stimata come:

$$\begin{aligned} &\approx O(N_1) \cdot O(100) \\ &\approx O(100 \cdot 100) \\ &= O(10^4) \end{aligned}$$

Pertanto, considerando una macchina che esegue circa 10^8 operazioni al secondo, il tempo stimato di esecuzione sarebbe:

$$\frac{10^4}{10^8} \text{ secondi} = 10^{-4} \text{ secondi.}$$

In conclusione, il programma è coerente con il limite di tempo imposto dal problema (1.00 s).

L'approccio *brute force* è, in generale, il più dispendioso dal punto di vista computazionale. In questo caso specifico, tuttavia, la dimensione ridotta degli input consente di rientrare nei limiti temporali previsti, permettendo al programma di terminare correttamente l'esecuzione.

Nella maggior parte dei casi, però, tale strategia non è applicabile e si rende necessario ricorrere a metodi algoritmici più efficienti.

Di seguito vengono presentati due problemi risolti mediante un approccio *brute force* che, nonostante la correttezza logica della soluzione, non rispettano i vincoli temporali imposti dal problema.

3.1.2 Ruota della fortuna

Link al problema

Time limit = 1.00s, Memory limit = 512 MB

Giorgio gestisce una ruota della fortuna a premi.

La ruota è suddivisa in N spicchi uguali, numerati da 0 a $N - 1$, in senso orario. Lo spicchio i corrisponde a un premio di valore V_i .

I premi vengono assegnati in questo modo. Dopo aver comprato il biglietto, ognuno dei giocatori si posiziona davanti allo spicchio di sua scelta e non si muove più. Quando la ruota viene fatta girare e si ferma, ciascun giocatore vince un premio in base al valore del nuovo spicchio che si trova davanti a lui.

Giorgio ha taroccato il meccanismo della ruota per fermarla quando gli conviene di più. Ti viene dato il valore dei premi per ogni spicchio, e il numero G_i di giocatori posizionati davanti a ciascuno spicchio i . Aiuta Giorgio a calcolare la somma totale dei premi che dovrà pagar, fermando la ruota in modo che questa somma sia minima!

Dati di input

La prima riga contiene un intero T , il numero di casi di test. Seguono T casi di test, numerati da 1 a T . Ogni caso di test è preceduto da una riga vuota.

Ogni caso test è composto da 3 righe:

- La prima riga contiene l'intero N .
- la seconda riga contiene il valore dei premi $V_0, \dots, V_{N-1}$ di ciascuno spicchio;
- la terza riga contiene il numero di giocatori $G_0, \dots, G_{N-1}$ che si sono posizionati in corrispondenza di ciascuno spicchio.

Dati di output

Il file di output deve contenere la risposta ai casi di test che sei riuscito a risolvere. Per ogni caso di test che hai risolto, il file di output deve contenere una riga con la dicitura:

Case #t: x

dove t è il numero del caso di test (a partire da 1) e il valore x è la somma totale dei premi minima.

Assunzioni e vincoli

- $T = 13$, nei file di input che scaricherai saranno presenti esattamente 13 casi di test.
- $2 \leq N \leq 1000$
- $0 \leq V_i \leq 1000$ per $i = 0, \dots, N - 1$
- $0 \leq G_i \leq 1000$ per $i = 0, \dots, N - 1$

Esempi di input/output

Input:

3

5

7 2 5 3 4

1 0 0 1 0

5

7 2 5 3 4

2 4 1 3 7

5

7 2 5 3 4

3 8 1 3 0

Output:

Case #1: 5

Case #2: 61

Case #3: 51

Spiegazione dell'esempio:

In tutti e tre i casi la ruota ha $N = 5$ spicchi con premi 7, 2, 5, 3 e 4.

Nel **primo caso di esempio**, ci sono solo due giocatori. La somma minima dei premi è 5, e si ottiene fermando la ruota ruotata di 2 spicchi in senso orario. In questo modo il primo giocatore riceverà un premio di 3 e il secondo un premio di 2.

Nel **secondo caso d'esempio**, la somma minima dei premi è 61, e si ottiene fermando la ruota ruotata di 3 spicchi in senso orario. (Quindi 2 giocatori riceveranno un premio da 5, 4 da 3, 1 da 4, 3 da 7, e 7 da 2. La somma totale dei premi in euro è data da $2 \cdot 5 + 4 \cdot 3 + 1 \cdot 4 + 3 \cdot 7 + 7 \cdot 2 = 61$).

Nel **terzo caso d'esempio**, la somma minima dei premi è 51, e si ottiene fermando la ruota esattamente nella posizione di partenza.

Idea (brute force)

Un approccio diretto al problema consiste nel fissare l'ordine di uno dei due vettori e provare tutte le possibili rotazioni dell'altro vettore. Per ogni rotazione si calcola il prodotto scalare tra i due vettori, sommando i prodotti degli elementi nelle stesse posizioni, e si mantiene il valore minimo ottenuto.

Implementazione C++ - Brute Force

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Funzione che ruota a destra di una posizione il vettore 'valori'
5 vector<int> rotate(vector<int> valori, int N){
6     int temp = valori[N-1]; // Salva l'ultimo elemento del vettore
```

```

7
8 // Shift a destra di tutti gli elementi
9 for(int i = N-1; i > 0; i--){
10     valori[i] = valori[i-1];
11 }
12
13 valori[0] = temp; // Inserisce l'ultimo elemento in prima
    posizione
14
15 return valori; // Restituisce il vettore ruotato
16 }
17
18 int main()
19 {
20     ifstream inputFile("fortuna_input_4.txt"); // Apertura del file di
        input
21
22     int T;
23     inputFile >> T; // Numero di casi di test
24
25     // Ciclo sui casi di test
26     for(int test = 1; test <= T; test++){
27         int ans = INT_MAX; // Valore minimo del prodotto scalare
            trovato
28         int temp_ans = 0; // Valore temporaneo del prodotto scalare
29
30         int N;
31         inputFile >> N; // Numero di elementi dei vettori
32
33         vector<int> V, G; // Vettori di input
34
35         // Lettura del vettore V
36         for(int i = 0; i < N; i++){
37             int v;
38             inputFile >> v; // Legge un elemento
39             V.push_back(v); // Inserisce l'elemento nel vettore V
40         }
41
42         // Lettura del vettore G
43         for(int i = 0; i < N; i++){
44             int g;
45             inputFile >> g; // Legge un elemento
46             G.push_back(g); // Inserisce l'elemento nel vettore G
47         }
48
49         // Prova tutte le N rotazioni del vettore V
50         for(int i = 0; i < N; i++){
51             temp_ans = 0; // Azzera la somma per la rotazione
                corrente
52
53             // Calcolo del prodotto scalare tra V e G
54             for(int j = 0; j < N; j++){
55                 temp_ans += G[j] * V[j];

```

```

56         }
57
58         V = rotate(V, N);           // Ruota il vettore V di una
           posizione
59         ans = min(temp_ans, ans); // Aggiorna il minimo trovato
60     }
61
62
63     cout << "Case #" << test << ": " << ans << endl;
64 }
65
66 return 0;
67 }

```

3.1.3 Stick Lengths

Link al problema

Time limit = 1.00s, Memory limit = 512 MB

There are n sticks with some lengths. Your task is to modify the sticks so that each stick has the same length.

You can either lengthen and shorten each stick. Both operations cost x where x is the difference between the new and original length.

What is the minimum total cost?

Input:

The first input line contains an integer n : the number of sticks.

Then there are n integers: $p_1, p_2, \dots, p_n$: the lengths of the sticks.

Output:

Print one integer: the minimum total cost.

Constraints:

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq p_i \leq 10^9$

Example:

Input:

5
2 3 1 5 2

Output:

5

Idea (brute force):

provare ogni possibile scelta di lunghezza target tra quelle presenti nei dati e calcolare il costo totale per uniformare tutti i bastoncini a quel valore; quindi scegliere il minimo tra tutti i costi calcolati. Nel caso d'esempio d'input, occorre calcolare quanto costa uniformare tutti i numeri a 1, poi a 2, poi a 3, poi a 4 ed infine a 5

Implementazione C++ - Brute Force

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     vector<int> sticks;           // Vettore che conterra' le lunghezze
        dei bastoncini
```

```

7      int numbers;                // Numero di bastoncini n
8      int costo_minimo = INT_MAX; // Variabile per memorizzare il costo
                                     minimo trovato, inizializzata all'infinito
9
10     cin >> numbers;             // Leggo n, il numero di bastoncini
11
12     // Ciclo per leggere le lunghezze dei bastoncini: O(n)
13     for (int i = 0; i < numbers; i++)
14     {
15         int stick;
16         cin >> stick;           // Leggo la lunghezza di un
                                     bastoncino
17         sticks.push_back(stick); // Inserisco la lunghezza nel vettore
18     }
19
20     // Calcolo il minimo e il massimo tra le lunghezze: O(n)
21     int minimo = *min_element(sticks.begin(), sticks.end());
22     int massimo = *max_element(sticks.begin(), sticks.end());
23
24     // Ciclo principale: provo ogni possibile lunghezza target tra
                                     minimo e massimo
25     // Complessita': O(R * n), dove R = massimo - minimo
26     for (int i = minimo; i <= massimo; i++)
27     {
28         int somma_costi = 0; // Somma dei costi per portare tutti i
                                     bastoncini alla lunghezza target i
29
30         // Per ogni bastoncino calcolo il costo per portarlo alla
                                     lunghezza target i: O(n)
31         for (int stick : sticks)
32         {
33             int costo = abs(stick - i); // Differenza tra lunghezza
                                     attuale e target
34             somma_costi += costo;       // Aggiungo il costo totale
35         }
36
37         // Aggiorno il costo minimo se la somma dei costi corrente
                                     inferiore
38         costo_minimo = min(costo_minimo, somma_costi);
39     }
40
41     cout << costo_minimo; // Stampo il costo totale minimo
42
43     return 0;
44 }

```

Costo Computazionale

Il costo computazionale è:

- $O(n)$ per il for della riga 13
- $O(n)$ per la funzione *min_element* della riga 21

- $O(n)$ per la funzione *max_element* della riga 22
- $O(\text{massimo} - \text{minimo}) \cdot O(n)$ per il for della riga 26 (incluso l'interno)

La complessità è:

$$\begin{aligned}
 &= O(n) + O(n) + O(n) + O(\text{massimo} - \text{minimo}) \cdot O(n) \\
 &= O(n) + O(R \cdot n) \\
 &= O(R \cdot n)
 \end{aligned}$$

dove: $R = \text{massimo} - \text{minimo}$, $n = \text{numero di bastoncini}$

Dai vincoli del problema:

$$1 \leq n \leq 2 \cdot 10^5, \quad 1 \leq p_i \leq 10^9,$$

quindi il valore massimo di R può arrivare fino a

$$R \leq 10^9.$$

Sostituendo nella complessità, otteniamo:

$$O(R \cdot n) = O(10^9 \cdot 2 \cdot 10^5) = O(2 \cdot 10^{14}).$$

Considerando una macchina che esegue circa 10^8 operazioni al secondo, il tempo stimato di esecuzione sarebbe:

$$\frac{2 \cdot 10^{14}}{10^8} = 2 \cdot 10^6 \text{ secondi} \approx 23 \text{ giorni!}$$

Pertanto, questo approccio è praticamente inefficiente e supera di gran lunga il time limit di 1 secondo.

Per risolvere il problema in tempo ragionevole, è necessario un algoritmo più efficiente, ad esempio basato sulla mediana dei bastoncini che vedremo in seguito.

3.1.4 Nearest Smaller Values

Link al problema

Time limit = 1.00s, Memory limit = 512 MB

Given an array of n integers, your task is to find for each array position the nearest position to its left having a smaller value.

Input:

The first input line has an integer n : the size of the array. The second line has n integers $x_1, x_2, \dots, x_n$: the array values.

Output

Print n integers: for each array position the nearest position with a smaller value. If there is no such position, print 0.

Constraints:

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq x_i \leq 10^9$

Example

Input:

8

2 5 1 4 8 3 2 5

Output:

0 1 0 3 4 3 3 7

Idea (brute force)

Dato un array di N numeri interi, per ogni posizione i si vogliono analizzare tutti gli elementi che si trovano alla sua sinistra (cioè con indice minore di i) per individuare un valore strettamente più piccolo di quello corrente. Tra tutti i valori più piccoli trovati, si seleziona quello più vicino alla posizione i , cioè quello con l'indice maggiore tra quelli validi.

Il programma utilizza due cicli annidati:

- il ciclo esterno scorre tutte le posizioni dell'array;
- il ciclo interno scorre le posizioni precedenti a quella corrente, confrontando i valori.

Ogni volta che viene trovato un valore più piccolo, la sua posizione viene salvata come possibile risposta. Poiché il ciclo interno procede da sinistra verso destra, l'ultima posizione salvata corrisponde alla più vicina a sinistra, che è quella richiesta dal problema.

Se per una determinata posizione non viene trovato alcun valore più piccolo, il risultato rimane pari a 0, indicando che non esiste una posizione valida alla sua sinistra.

Implementazione C++ - Brute Force:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int64_t N;          // Numero di elementi dell'array
7      cin >> N;          // Lettura di N da input
8
9      // array      : contiene i valori dell'array
10     // results    : contiene la risposta per ogni posizione,
11     //              inizializzata a 0 (nessun elemento pi piccolo a
12     //              sinistra)
13
14     vector<int> array, results(N, 0);
15
16     // Lettura dei valori dell'array
17     for (int i = 0; i < N; i++)
18     {
19         int64_t x;        // Variabile temporanea per leggere ogni valore
20         cin >> x;         // Lettura del valore
21         array.push_back(x); // Inserimento in coda al vector
22     }
23
24     // Per ogni posizione i dell'array
25     for (int i = 0; i < N; i++)
26     {
27         // Scorriamo tutti gli elementi a sinistra di i
28         for (int j = 0; j < i; j++)
29         {
30             // Se troviamo un valore pi piccolo di array[i]
31             if (array[j] < array[i])
32             {
33                 // Aggiorniamo la risposta con la posizione (1-based)
34                 /* IMPORTANTE: Non interrompiamo il ciclo. alla fine
35                 rimarrà
36                 l'ULTIMA posizione valida trovata (la pi vicina a i)
37                 */
38                 results[i] = j + 1;
39             }
40         }
41     }
42
43     // Stampa delle soluzioni
44     for (int result : results)
45     {
46         cout << result << " ";
47     }
48
49     return 0;
50 }
```

Costo Computazionale

Il costo computazionale di questo programma può essere analizzato come segue:

- $O(N)$ per il ciclo for della riga 16.
- $O(i) = O(N - 1)$ per il ciclo for della riga 27 (la i arriva fino a $N - 1$).

Il costo computazionale è quindi:

$$= O(N) \cdot O(N - 1) = O(N) \cdot O(N) = O(N^2)$$

Questo significa che il numero di operazioni cresce quadraticamente con la dimensione dell'array. Considerando però che, dai vincoli del problema, $N \leq 2 \cdot 10^5$, allora quando N diventa troppo grande si ha:

$$O((10^5)^2) = O(10^{10})$$

che è superiore alle 10^8 operazioni che la nostra macchina può fare in 1 secondo. Occorre quindi trovare una strategia diversa per poter risolvere il problema nei limiti imposti.

3.2 Idee Greedy

Gli algoritmi *greedy* si basano sull'idea di compiere, a ogni passo, la scelta che risulta immediatamente più conveniente, senza considerare in modo esplicito le conseguenze future. La soluzione viene quindi costruita progressivamente, fissando le decisioni man mano che vengono prese e senza possibilità di modificarle successivamente.

Questo approccio consente spesso di ottenere algoritmi semplici ed efficienti, caratterizzati da una bassa complessità computazionale. Tuttavia, la sua applicabilità non è universale: una soluzione greedy è corretta solo se il problema ammette una struttura tale per cui una scelta ottimale a livello locale conduce anche a una soluzione ottimale globale. In assenza di questa proprietà, l'algoritmo può produrre risultati non corretti, pur risultando computazionalmente rapido.

In questo contesto, l'analisi dei controesempi riveste un ruolo fondamentale. Un singolo controesempio è infatti sufficiente a dimostrare la non correttezza di una strategia greedy, mostrando come una scelta localmente ottimale possa condurre a una soluzione globale subottimale. Per questo motivo, è importante imparare a costruire esempi che mostrino come un algoritmo greedy possa fallire, evidenziandone così la non validità.

Esempio di idea Greedy:

Supponiamo di avere una rete di piazze collegate tra loro da strade. In ogni piazza vi è un certo quantitativo di monete a terra che possono essere raccolte solo se si passa per quella determinata piazza.

Bisogna costruire un algoritmo che determini il percorso che permetta di raccogliere il maggior numero di monete.

Con riferimento alla figura 3.1, partendo dalla piazza M_1 , posso scegliere tra due percorsi:

- $M_1 \rightarrow M_2 \rightarrow M_3$
- $M_1 \rightarrow M_4 \rightarrow M_5$

Qual è il percorso più redditizio?

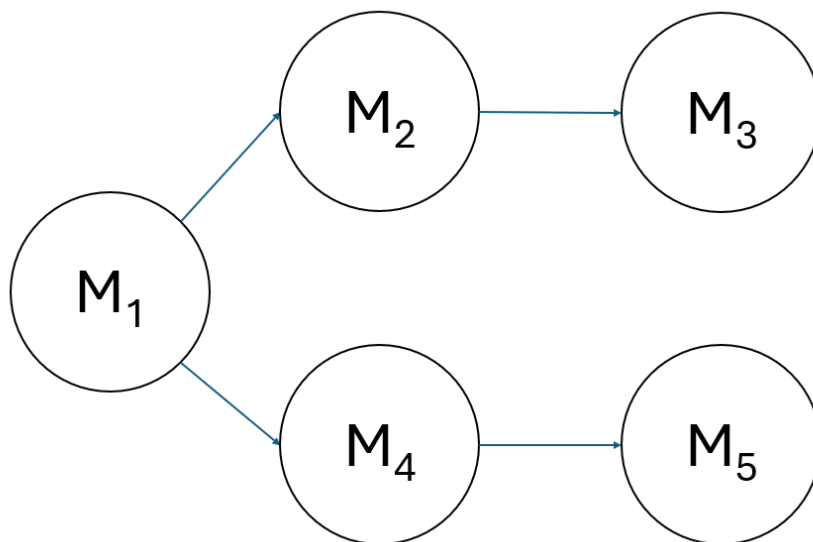


Figura 3.1: Percorso con piazze (cerchi) e strade (freccie). Con M_i sono indicate le monete presenti in ogni piazza

L'approccio **Brute Force** consiste nel percorrere tutte le strade e, quindi, valutare il massimo tra:

- $M_1 + M_2 + M_3$
- $M_1 + M_4 + M_5$

Ciò garantisce la correttezza della soluzione pagando in termini di complessità computazionale: al crescere delle possibili strade, l'algoritmo diventa insostenibile in termini di costo.

Un approccio **Greedy**, invece, consiste nel prendere, ad ogni piazza, la scelta immediatamente più conveniente, cioè seguire la strada che conduce alla piazza con il maggior numero di monete tra quelle disponibili. In altre parole, a ogni nodo del grafo, si seleziona localmente il percorso che massimizza la quantità di monete raccolte in quel passo, senza considerare tutte le possibili combinazioni future.

Con riferimento alla figura 3.1, partendo dalla piazza M_1 , la scelta greedy consiste nel confrontare le piazze raggiungibili (M_2 e M_4) e scegliere immediatamente quella con il maggior numero di monete. Successivamente, si ripete lo stesso criterio per la piazza successiva fino a raggiungere una piazza terminale.

Come detto, l'approccio si concretizza nel fare la scelta migliore *localmente*, senza considerare le conseguenze future.

Questo metodo è computazionalmente efficiente, in quanto richiede solo di confrontare i valori locali delle piazze accessibili in ogni passo. Tuttavia, *non sempre garantisce il massimo globale*, poiché una scelta ottimale locale potrebbe portare a un percorso complessivamente subottimale. Per questo motivo, la strategia greedy va applicata solo quando il problema possiede una struttura che assicuri la correttezza delle scelte locali rispetto alla soluzione globale, oppure come heuristica rapida in problemi di grandi dimensioni. A tal proposito, in figura 3.2 è riportato un controesempio che smonta la scelta greedy sopra proposta.

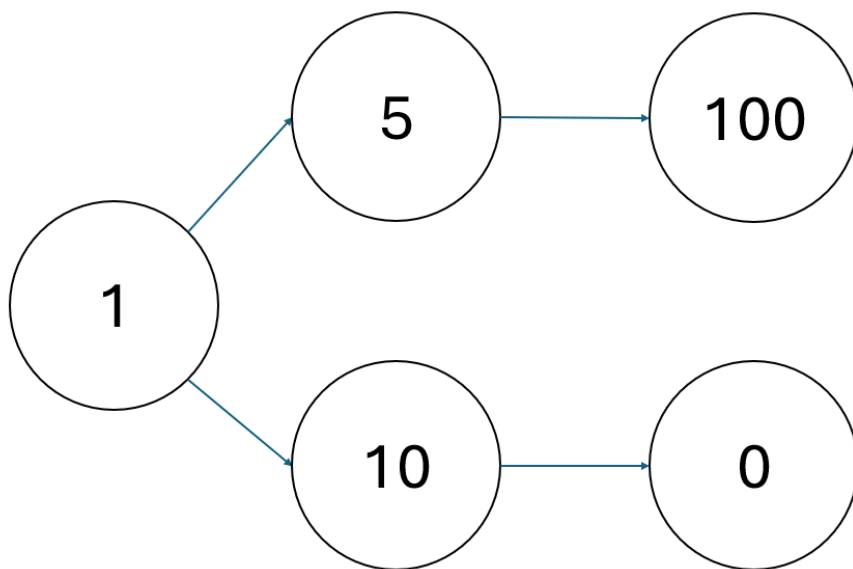


Figura 3.2: Controesempio: scegliendo al primo passo la piazza che garantisce il massimo immediato (10 monete), il percorso totale raccoglie solo 11 monete, mentre scegliendo l'altra strada si potrebbero raccogliere 106 monete. Questo evidenzia come la scelta greedy locale possa portare a un risultato subottimale.

3.2.1 Calcolatrice Rotta

Link al problema

Francesco ha fatto cadere la sua calcolatrice, e ora non funziona più come dovrebbe! Gli unici tasti funzionanti sono il $-$, il $\times$, l'1 e il 2.

Per utilizzare la calcolatrice è costretto a partire dal numero 1 o dal numero 2 (premendo il tasto corrispondente) e applicare zero o più volte una delle 4 possibili operazioni funzionanti:

- sottrarre 1
- sottrarre 2
- moltiplicare per 1
- moltiplicare per 2

Per fare uno scherzo ai suoi amici, vorrebbe raggiungere sulla calcolatrice il numero N . Quante operazioni deve fare al minimo per farlo?

Nota: *premere il tasto 1 o 2 all'inizio conta come un'operazione, inoltre la calcolatrice è danneggiata quindi Francesco è costretto a partire sempre da 1 o 2 (non può ad esempio premere 2 e poi 1 per partire dal numero 21) e le uniche operazioni ammesse sono quelle precedentemente elencate (non può per esempio moltiplicare o sottrarre 12 o 21).*

Formato di input

La prima riga del file di input contiene un intero T , il numero di casi di test. Seguono T casi di test, numerati da 1 a T . Ogni caso di test è preceduto da una riga vuota.

Ogni caso di test è composto come segue:

- una riga contenente l'intero N .

Formato di output

Il file di output deve contenere la risposta ai casi di test che sei riuscito a risolvere. Per ogni caso di test risolto, il file di output deve contenere una riga con la dicitura:

Case #test: operazioni

dove **test** è il numero del caso di test (a partire da 1) e **operazioni** è la risposta al caso di test: il minimo numero di operazioni necessarie per raggiungere N .

Assunzioni

- $T = 10$, nei file di input che scaricherai saranno presenti esattamente 10 casi di test.
- $1 \leq N \leq 10^{18}$

Esempio

Input:

2

2

5

13

Output:

Case #1: 1

Case #2: 5

Case #3: 6

Idea 1 (greedy)

Una prima idea greedy potrebbe essere quella di imporre come soluzione 1 se il numero da raggiungere è 1 o 2 (infatti basta premere il corrispondente tasto). Se il numero da raggiungere dovesse essere più alto, allora si può partire da 2 e raddoppiare il numero fin quando non si supera il numero obiettivo. Una volta superato, si sottrae 1 fino ad arrivare al numero obiettivo.

Controesempio: La strategia greedy descritta non garantisce una soluzione ottimale. Ad esempio, per raggiungere il valore 20, l'approccio porta al seguente percorso:

$$2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 31 \rightarrow 30 \rightarrow \dots \rightarrow 20$$

che richiede complessivamente 17 operazioni.

Tuttavia, è possibile raggiungere lo stesso valore obiettivo con un numero minore di operazioni seguendo il percorso:

$$2 \rightarrow 4 \rightarrow 8 \rightarrow 7 \rightarrow 6 \rightarrow 5 \rightarrow 10 \rightarrow 20$$

che utilizza esclusivamente operazioni ammesse e richiede soltanto 7 operazioni. Questo controesempio dimostra che la strategia detta non è ottimale.

Idea 2 (greedy)

Una seconda idea potrebbe essere quella di partire dal numero obiettivo e dividere per 2 se il numero è pari, aggiungere 1 se il numero è dispari. Così facendo, il percorso inverso sarebbe:

$$20 \rightarrow 10 \rightarrow 5 \rightarrow 6 \rightarrow 3 \rightarrow 4 \rightarrow 2$$

Così facendo, il percorso reale sarebbe:

$$2 \rightarrow 4 \rightarrow 3 \rightarrow 6 \rightarrow 5 \rightarrow 10 \rightarrow 20$$

Ciò garantisce il minimo numero di operazioni senza controesempi!

Implementazione C++ - Greedy:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      ifstream inputFile("calcolatrice_input_1.txt");
7
8      int T;
9      inputFile >> T;
10
11     for (int test = 1; test <= T; ++test)
12     {
13
14         int64_t N;
15         inputFile >> N;
16
17         int64_t operazioni = 1;
18
19         if(N==1 || N == 2){
20             operazioni = 1;
21         }
22         else if(N == 0){
23             operazioni = 2;
24         }
25         else{
26
27             /*
28              Parto dal numero obiettivo e divido per 2 se il numero
29              pari
30              sommo 1 se il numero    dispari
31              */
32             while(N > 2){
33                 if(N%2 == 0){ //se il numero    pari
34                     N = N / 2; //divido per 2
35                     operazioni += 1;
36                 }
37                 else{ //se il numero    dispari
38                     N = N + 1; //sommo 1
39                     operazioni += 1;
40                 }
41             }
42
43             cout << "Case #" << test << ": ";
44             cout << operazioni << endl;
45         }
46
47         inputFile.close();
48
49         return 0;
50     }
```

3.2.2 Precise Average

Link al Problema

Time limit = 1.00s, Memory limit = 512 MB

Your friend John works at a shop that sells N types of products, numbered from 0 to $N-1$. Product i has a price of P_i bytedollars, where P_i is a positive integer. The government of Byteland has created a new law for shops. The law states that the average of the prices of all products in a shop should be exactly K , where K is a positive integer. John's boss gave him the task to change the prices of the products so the shop would comply with the new law. He has lots of other stuff to do, so he asked you for help: what is the minimum number of products whose prices should be changed? A product's price can be changed to any positive integer amount of bytedollars.

Input

The first line of the input contains two integers N and K . The next line contains N positive integers, $P_0, \dots, P_{N-1}$.

Output

Output a single integer between 0 and N , inclusive: the answer to the question. It can be proven that it is always possible to change the prices of some products so that the new prices comply with the law.

Constraints:

- $1 \leq N \leq 200\,000$
- $1 \leq K \leq 1\,000\,000$
- $1 \leq p_i \leq 1\,000\,000$ for each $i = 0 \dots N-1$

Idea (Greedy)

L'idea che risolve il problema consiste nel fare una trattazione matematica per poter capire di quanto bisogna ritoccare la somma dei prezzi P_i al fine di arrivare alla media precisa K .

Considerando che la somma iniziale è:

$$S = \sum_{i=0}^{N-1} P_i$$

e la corrispondente media è:

$$M = \frac{\sum_{i=0}^{N-1} P_i}{N}$$

Lo scopo è quindi quello di trovare la correzione da effettuare alla somma (X), tale per cui la nuova media diventa esattamente K :

$$K = \frac{S - X}{N}$$

Da cui ricaviamo la correzione da fare:

$$X = S - K \cdot N$$

Dobbiamo quindi sottrarre o aggiungere (a seconda del segno) X ai prodotti P_i . In particolare, si verificano 3 casi:

- Se la media obiettivo è superiore rispetto a quella iniziale, sarà sufficiente aggiungere X ad un qualsiasi prodotto (quindi il numero minimo di prodotti da modificare è 1).
- Se la media obiettivo coincide con quella iniziale, non sarà necessario modificare alcun prodotto.
- Se la media obiettivo è inferiore alla media calcolata, sarà necessario sottrarre X dal totale dei prezzi dei prodotti. In particolare, ordinando il *vector* in ordine crescente, possiamo cominciare a sottrarre dall'elemento più grande (indice $N1$) fino a portarlo al valore minimo di 1, e procedere così con gli altri elementi fino a raggiungere la sottrazione totale di X .

Implementazione C++ - Greedy

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      // N = numero di prodotti
7      // K = valore della media desiderata
8      int64_t N, K;
9      cin >> N >> K;
10
11     // Somma iniziale di tutti gli elementi
12     int64_t somma_iniziale = 0;
13
14     // Vettore dei valori
15     vector<int> P(N);
16     for (int i = 0; i < N; ++i)
17     {
18         cin >> P[i];
19         somma_iniziale += P[i];          // Aggiorna la somma totale
20     }
21
22     // Ordiniamo i valori in ordine crescente
23     // Ci servir per agire prima sugli elementi pi grandi
24     sort(P.begin(), P.end());
25
26     // Risposta finale: numero minimo di elementi da modificare
27     int64_t ans = 0;
28
29     // "correzione" rappresenta di quanto la somma attuale
30     // supera la somma ideale K * N
31     // Se e' positiva, dobbiamo ridurre la somma
32     int64_t correzione = somma_iniziale - K * N;
33
34     // Caso 1: la somma e' gia' minore di K*N

```

```

35 // Basta modificare solo un elemento
36 if (correzione < 0)
37 {
38     ans = 1;
39 }
40 // Caso 2: la media e' gia' esattamente K
41 // Non serve alcuna modifica
42 else if (correzione == 0)
43 {
44     ans = 0;
45 }
46 // Caso 3: la somma e' troppo grande e va ridotta
47 else
48 {
49     // Partiamo dall'elemento piu' grande
50     for (int i = N - 1; i >= 0; i--)
51     {
52         // P[i], se minore di correzione, puo' essere diminuito
53         // fino ad 1.
54         if (P[i] <= correzione && P[i] > 1)
55         {
56             // Dopo aver abbassato P[i], calcolo la correzione
57             // residua
58             correzione -= P[i] - 1;
59
60             // Aumento il numero di elementi modificati
61             ans++;
62         }
63         // Se P[i] e' maggiore della correzione residua
64         // basta modificare solo questo elemento
65         else if (P[i] > correzione)
66         {
67             ans++; //Basta diminuire solo P[i] dell'intera
68             // correzione
69             break; // La correzione e' risolta
70         }
71         // Se l'elemento e' gia' 1, non possiamo ridurlo e passiamo
72         // al prossimo
73         else if (P[i] == 1)
74         {
75             continue;
76         }
77     }
78 }
79
80 // Stampa il numero minimo di modifiche necessarie
81 cout << ans << endl;
82
83 return 0;
84 }

```

Capitolo 4

Pattern Algoritmici

In questo capitolo verranno introdotti alcuni pattern algoritmici classici utilizzati nella risoluzione efficiente dei problemi computazionali. Queste tecniche rappresentano strategie ricorrenti nella progettazione degli algoritmi e consentono di ridurre la complessità computazionale, migliorando le prestazioni rispetto a soluzioni brute force.

In particolare, verranno analizzati i seguenti pattern:

- **Sliding Window**, una strategia basata su finestre mobili per l'elaborazione di sottoarray o sottostringhe contigue;
- **Bitmask**, una rappresentazione compatta degli insiemi che consente di gestire combinazioni e stati in modo efficiente tramite operazioni sui bit.
- **Prefix Sum**, una tecnica di preprocessing che permette di rispondere rapidamente a interrogazioni su intervalli;

Lo studio di questi pattern fornisce una base fondamentale per affrontare problemi di programmazione competitiva e per comprendere concetti più avanzati della progettazione algoritmica.

4.1 Sliding Window

La tecnica della *Sliding Window* (finestra mobile) è una strategia algoritmica utilizzata per analizzare sottoarray o sottostringhe contigue di una sequenza di dati in modo efficiente. Essa si basa sul mantenimento di una finestra che scorre progressivamente lungo la struttura dati, evitando ricalcoli ridondanti e consentendo di ridurre la complessità computazionale rispetto a soluzioni di tipo brute force.

L'idea principale consiste nel definire due indici che delimitano l'inizio e la fine della finestra e nell'aggiornare dinamicamente le informazioni associate alla finestra corrente (come somme, conteggi o massimi) man mano che la finestra viene spostata.

A seconda del problema, la dimensione della finestra può essere:

- *fissa*, quando la finestra ha lunghezza costante;
- *variabile*, quando la finestra si espande o si contrae in base a determinate condizioni.

Grazie a questo approccio, molti problemi che richiederebbero un tempo di esecuzione quadratico $O(n^2)$ possono essere risolti in tempo lineare $O(n)$. La tecnica della Sliding Window è ampiamente utilizzata in ambiti quali l'elaborazione di array, stringhe e flussi di dati, e rappresenta uno strumento fondamentale nella progettazione di algoritmi efficienti.

Sliding Window fissa

Supponiamo di avere un **vector** $A = (A_0, A_1, \dots, A_{N-1})$ e di voler determinare la somma massima degli elementi appartenenti a sottoarray contigui di lunghezza fissa k .

Un approccio ingenuo consiste nel calcolare la somma di tutti i possibili sottoarray di lunghezza k , ottenendo una complessità computazionale pari a $O(N \cdot k)$. Tale soluzione risulta inefficiente per valori elevati di N e k .

Il codice di questo approccio brute force è il seguente:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int N, k;
6      cin >> N >> k;
7
8      vector<int> A(N);
9      for (int i = 0; i < N; i++) {
10         cin >> A[i];
11     }
12
13     int maxSum = INT_MIN;
14
15     // ciclo su tutte le finestre
16     for (int i = 0; i <= N - k; i++) {
17         int currentSum = 0;
18
19         // calcolo la somma della finestra [i, i+k-1]
20         for (int j = i; j < i + k; j++) {
21             currentSum += A[j];
22         }
23
24         maxSum = max(maxSum, currentSum);
25     }
26
27     cout << maxSum << endl;
28
29     return 0;
30 }
```

La tecnica della *Sliding Window* consente di risolvere il problema in tempo lineare $O(N)$ mantenendo una finestra di dimensione costante k che scorre lungo il vettore. Inizialmente si calcola la somma dei primi k elementi. Successivamente, la finestra viene spostata di una posizione verso destra sottraendo l'elemento che esce dalla finestra e aggiungendo l'elemento che entra (figura 4.1).

Indicando con $[left, right]$ gli estremi della finestra, con $right - left + 1 = k$, l'aggiornamento della somma può essere espresso come:

$$S_{\text{nuovo}} = S_{\text{precedente}} - A_{\text{left}} + A_{\text{right}+1}$$

Dove con A_{left} intendiamo l'elemento che esce dalla finestra dopo il suo scorrimento, $A_{\text{right}+1}$ intendiamo l'elemento che entra nella finestra.

Ripetendo tale procedura fino a raggiungere la fine del vettore, è possibile determinare la somma massima tra tutte le finestre considerate.

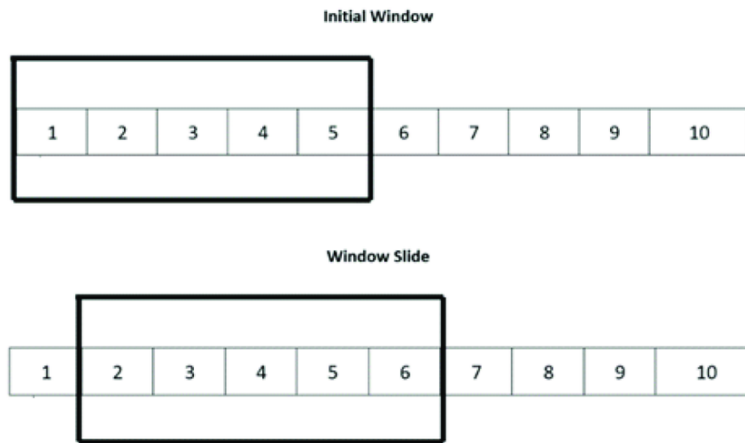


Figura 4.1: Sliding Window fissa con $k = 5$. In questo caso, per ogni aumento del margine destro (*right*) vi è anche un aumento del margine sinistro (*left*)

La codifica con la Sliding Window fissa è la seguente:

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int N, k;
6      cin >> N >> k;
7
8      vector<int> A(N);
9      for (int i = 0; i < N; i++)
10         cin >> A[i];
11
12     int maxSum = INT_MIN;
13     int windowSum = 0;
14     int left = 0;
15
16     for (int right = 0; right < N; right++) {
17         windowSum += A[right]; // aggiungo l'elemento entrante
18
19         // se la finestra supera k elementi, rimuovo quello uscente
20         if (right - left + 1 > k) {
21             windowSum -= A[left];
22             left++;
23         }
24
25         // aggiorno il massimo
26         if (right - left + 1 == k)
27             maxSum = max(maxSum, windowSum);
28     }
29
30     cout << maxSum << endl;
31
32     return 0;
33 }
```

Questo approccio evita il ricalcolo completo della somma per ogni sottoarray e permette di ridurre la complessità temporale da $O(N \cdot k)$ a $O(N)$.

Sliding Window variabile

Supponiamo di avere un **vector** $A = (A_0, A_1, \dots, A_{N-1})$ e di voler determinare la massima somma di un sottoarray contiguo tale che la somma non superi un certo valore M (oppure soddisfi un'altra condizione definita dal problema).

In questo caso, la lunghezza della finestra non è più fissa, ma varia dinamicamente durante lo scorrimento.

Ancora una volta, l'approccio ingenuo consiste nel verificare tutte le possibili finestre, ottenendo una complessità pari a $O(N^2)$, che risulta inefficiente per valori elevati di N .

Il codice brute force è analogo al caso precedente, ma con due cicli annidati su tutti i possibili intervalli.

La tecnica della *Sliding Window variabile* consente di risolvere il problema in tempo lineare $O(N)$ mantenendo una finestra definita dagli indici $[left, right]$. Si procede nel seguente modo:

- L'indice *right* scorre l'intero array aggiungendo gli elementi alla somma della finestra.
- Quando la somma o la condizione della finestra non è più soddisfatta, si sposta *left* verso destra rimuovendo gli elementi usciti dalla finestra.
- Ad ogni passo si aggiorna il valore massimo o la quantità desiderata.

La codifica in C++ è la seguente:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int N, M;
6     cin >> N >> M;    // N elementi e somma massima consentita
7
8     vector<int> A(N);
9     for (int i = 0; i < N; i++){
10         cin >> A[i];
11     }
12
13
14     int maxSum = 0;
15     int windowSum = 0;
16     int left = 0;
17
18     for (int right = 0; right < N; right++) {
19         windowSum += A[right];    // aggiungo l'elemento entrante
20
21         // riduco la finestra finche' la condizione non è soddisfatta
22         while (windowSum > M && left <= right) {
23             windowSum -= A[left];
24             left++;
25         }
26
27         // aggiorno il massimo
28         maxSum = max(maxSum, windowSum);
29     }
```

```

29     }
30
31     cout << maxSum << endl;
32
33     return 0;
34 }

```

In questo caso:

1. *right* espande la finestra includendo nuovi elementi.
2. *left* contrae la finestra quando la condizione non è più rispettata.
3. *maxSum* viene aggiornato solo quando la finestra soddisfa la condizione.
4. Complessità: $O(N)$, perché ogni elemento entra ed esce dalla finestra al massimo una volta.

La figura 4.2 illustra il meccanismo di espansione e contrazione della sliding window.

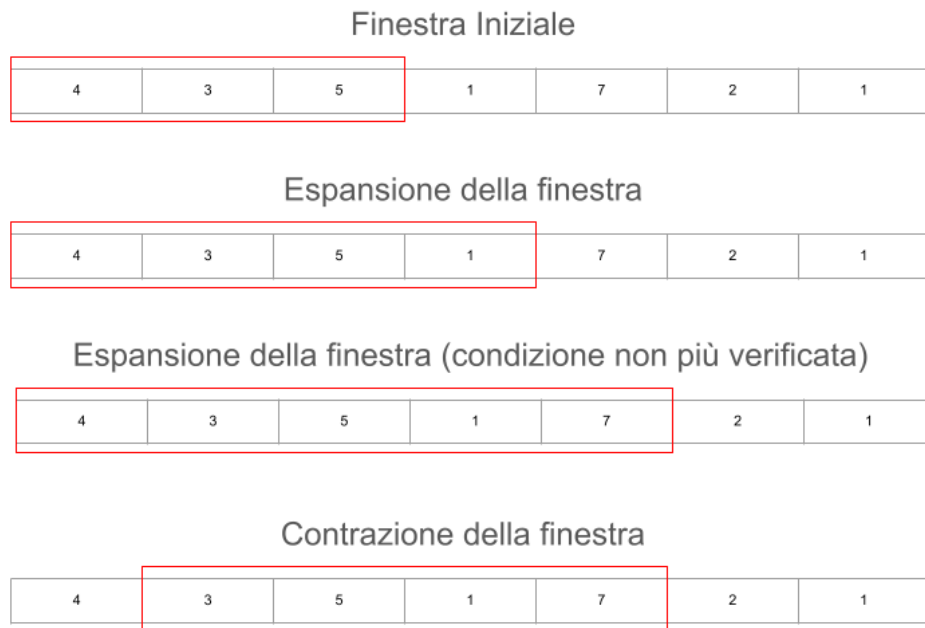


Figura 4.2: Sliding Window variabile. I margini *left* e *right* sono indipendenti e vengono gestiti in maniera che la finestra rispetti una certa condizione

Questo schema è molto utilizzato per problemi con vincoli dinamici sulla finestra, come:

- somma massima/minima sotto un certo limite,
- conteggio massimo di elementi con una proprietà,
- lunghezza massima di sottosequenze valide, ecc.

4.1.1 Christmas Light

Link al problema

Time limit = 1.00s, Memory limit = 512 MB

Davide wants to decorate his home for Christmas, even though his budget as a student is pretty limited, and he needs to avoid excessive expenses. After days of searching around the city, he finally found a nice row of almost new Christmas lights: it is time to start the decorations!

The row consists of N consecutive lights, numbered from 0 to $N - 1$. Each light has a colour L_i ($i = 0 \dots N - 1$), represented by a number between 0 and $C - 1$. Every colour between 0 and $C - 1$ appears at least once in the row of lights. In order to save energy (running the house dynamo is tiring!), he wants to cut off a segment of consecutive lights from the row containing every colour between 0 and $C - 1$ at least once. What is the length of the shortest segment he can cut?

Input

The first line contains the two integers N and C . The second line contains N integers L_i .

Output

You need to write a single line with an integer: the length of a shortest segment of lights containing every colour.

Constraints

- $3 \leq C \leq N \leq 200000$.
- $0 \leq L_i < C$ for each $i = 0 \dots N - 1$.
- Every colour c between 0 and $C - 1$ appears at least once in the row of lights, i.e., there exists at least one i ($0 \leq i < N$) such that $L_i = c$.

Examples

Input	Output
10 4 0 1 3 0 0 1 2 0 2 3	5
11 3 1 1 0 0 2 2 1 1 1 2 2	4

È consigliato visionare il link al problema ([clicca qui](#)) per comprendere al meglio il significato dell'output.

Idea (Sliding Window)

Si considera la fila di N luci come un array L di lunghezza N , in cui ogni elemento rappresenta il colore della luce corrispondente. L'obiettivo è individuare un segmento di luci consecutive contenente almeno una volta ogni colore tra 0 e $C - 1$.

Utilizziamo quindi una sliding window per esplorare tutti i segmenti possibili in maniera efficiente. Si mantengono due indici, *left* e *right*, che rappresentano rispettivamente l'inizio e la fine della finestra corrente.

Procedura generale:

- Si estende la finestra aumentando *right* e aggiornando un array *count* che tiene traccia del numero di occorrenze di ciascun colore all'interno della finestra.
- Si aggiorna una variabile *total_color* che indica quanti colori distinti sono presenti nella finestra.
- Quando *total_color* raggiunge C , significa che la finestra contiene tutti i colori almeno una volta. In questo caso, si prova a restringere la finestra spostando *left* verso destra, aggiornando di volta in volta la lunghezza minima della finestra.

In questo modo si esplorano tutti i segmenti validi contenenti tutti i colori, senza dover controllare tutte le possibili sotto-sequenze. La lunghezza minima trovata durante il processo rappresenta la soluzione del problema.

Implementazione C++:

```
1 // https://training.olinfo.it/task/ois_lights
2
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 int main()
7 {
8     int64_t N, C;
9     cin >> N;
10    cin >> C;
11
12    vector<int> count(C, 0); // count e' il vettore che contiene quante
13                               luci accese ci sono per ogni colore. Nella posizione 1, ad
14                               esempio, se c'è il numero 2, vuol dire che ci sono 2 luci
15                               accesi del colore 1.
16    vector<int64_t> L(N);
17
18    for (int i = 0; i < N; i++)
19    {
20        cin >> L[i];
21    }
22
23    // Metodo della sliding window credo
24
25    int left = 0, right = 0; //finestra iniziale
26    int total_color = 0;
27    int dist = INT_MAX;
```

```

26   for (right = 0; right < N; right++) //il for praticamente sposta
    automaticamente la finestra a destra
27   {
28       count[L[right]]++;
29
30       if (count[L[right]] == 1)
31       {
32           total_color++;
33       }
34
35       while (total_color == C) // nella sliding window variabile il
    while e' un classico
36       // quando rispettata la condizione (o non rispettata,
37       // a seconda del problema), si riduce la finestra con "left++"
38       {
39           dist = min(dist, right - left + 1);
40
41
42           count[L[left]]--; // stiamo per restringere la finestra,
    quindi andiamo a togliere il colore corrispondente
43           // all'elemento uscente.
44
45           if(count[L[left]] == 0){ // se il contatore relativo a quel
    colore arriva a 0, allora il numero totale
46           // di colori diminuisce.
47               total_color--;
48           }
49
50           left++; // contrazione della finestra.
51
52       }
53
54   }
55
56   cout << dist << endl;
57
58   return 0;
59 }

```

4.1.2 Playlist

4.1.3 Distinct Values Subarray

4.2 Bitmask

La tecnica del *bitmask* è un metodo che consente di rappresentare sottoinsiemi di un insieme finito utilizzando la rappresentazione binaria degli interi. Ogni bit di un numero intero viene associato a un elemento dell'insieme: il valore 1 indica la presenza dell'elemento nel sottoinsieme, mentre il valore 0 ne indica l'assenza.

Dato un insieme di n elementi $\{e_0, e_1, \dots, e_{n-1}\}$, ogni sottoinsieme può essere rappresentato tramite un intero compreso tra 0 e $2^n - 1$. Incrementando progressivamente di una unità tale valore (da 0 fino a $2^n - 1$) e osservandone la rappresentazione binaria, si ottengono tutte le pos-

sibili combinazioni di presenza e assenza degli elementi, ovvero tutti i sottoinsiemi dell'insieme dato.

La tecnica del bitmask può essere vista come una forma di bruteforce sui sottoinsiemi; tuttavia, grazie alla rappresentazione compatta degli stati e all'uso delle operazioni bitwise, consente un'implementazione più efficiente e leggibile rispetto a un approccio ingenuo.

Essa è particolarmente utile nei problemi di programmazione competitiva e di progettazione di algoritmi in cui è necessario:

- generare tutti i sottoinsiemi di un insieme;
- verificare rapidamente l'appartenenza di un elemento a un insieme;
- ridurre l'uso della memoria rispetto a strutture dati più complesse.

4.2.1 Apple Division

Link al problema

Time limit = 1.00s, Memory limit = 512 MB

There are n apples with known weights. Your task is to divide the apples into two groups so that the difference between the weights of the groups is minimal.

Input

The first input line has an integer n : the number of apples. The next line has n integers $p_1, p_2, \dots, p_n$: the weight of each apple.

Output

Print one integer: the minimum difference between the weights of the groups.

Constraints

- $1 \leq n \leq 20$
- $1 \leq p_i \leq 10^9$

Example

Input

```
5
3 2 7 4 1
```

Output

```
1
```

Explanation: Group 1 has weights 2, 3 and 4 (total weight 9), and group 2 has weights 1 and 7 (total weight 8).

Idea (Bitmask)

Si associa alle N mele un array binario di lunghezza N , comunemente chiamato *bit mask*, che viene incrementato di 1 a ogni iterazione. In questo modo la bit mask assume, in successione, tutti i possibili valori binari:

```
000001
000010
000011
000100
000101
...
111111
```

Ogni configurazione rappresenta una possibile suddivisione delle mele in due gruppi: se il bit in posizione i vale 1, la mela i -esima viene assegnata al primo gruppo; se vale 0, viene assegnata al secondo gruppo.

Scorrendo tutte le possibili bit mask, si esplorano tutte le combinazioni possibili di suddivisione delle mele. Per ciascuna configurazione si calcola la differenza tra la somma delle mele dei due gruppi, scegliendo infine quella che minimizza tale differenza.

Al termine dell'esplorazione di tutte le possibili bitmask, il valore minimo ottenuto rappresenta la soluzione del problema.

Implementazione C++:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  /*
5   Funzione che calcola la somma delle mele appartenenti al gruppo
6   rappresentato dalla bitmask apple_bit.
7   Se apple_bit[i] == 1, allora apples[i] viene sommato.
8   */
9  int64_t somma_apple(int N, vector<bool> apple_bit, vector<int64_t> &
10     apples)
11  {
12     int64_t somma = 0;
13
14     // Scorre tutte le N posizioni della bitmask
15     for (int i = 0; i < N; i++)
16     {
17         // Se il bit vale 1, la mela i-esima appartiene al gruppo
18         if (apple_bit[i])
19         {
20             somma += apples[i];
21         }
22     }
23     return somma;
24 }
25
26 int main()
27 {
28     int N;
```

```

29     cin >> N;    // Numero di mele
30
31     vector<int64_t> apples;           // Vettore contenente il
        numero di mele in ogni posizione
32     vector<bool> apple_bit(N, 0);    // Bitmask iniziale: tutti 0 (
        tutte le mele nel secondo gruppo)
33
34     int64_t somma_tot = 0;           // Somma totale delle mele
35     int64_t somma_provvisoria = 0;   // Somma del gruppo
        selezionato dalla bitmask corrente
36     int64_t diff = INT_MAX;          // Differenza minima trovata (
        inizialmente molto grande)
37
38     // Lettura dei valori delle mele
39     for (int i = 0; i < N; i++)
40     {
41         int64_t apple;
42         cin >> apple;
43         apples.push_back(apple);
44     }
45
46     // Calcolo della somma totale delle mele
47     for (int apple : apples)
48     {
49         somma_tot += apple;
50     }
51
52     /*
53     Ciclo che genera tutte le possibili bitmask incrementando
54     il "numero binario" rappresentato da apple_bit.
55     */
56     while (somma_provvisoria < somma_tot)
57     {
58         // Simula l'incremento di un numero binario partendo dal bit
        meno significativo
59         for (int i = N - 1; i >= 0; i--)
60         {
61             if (apple_bit[i] == 0)
62             {
63                 // Se il bit 0, lo porto a 1 e mi fermo
64                 apple_bit[i] = 1;
65
66                 // Calcolo la somma del gruppo rappresentato dalla
        bitmask
67                 somma_provvisoria = somma_apple(N, apple_bit, apples);
68
69                 // Calcolo la differenza tra i due gruppi:
70                 // gruppo1 = somma_provvisoria
71                 // gruppo2 = somma_tot - somma_provvisoria
72                 diff = min(diff, abs(somma_tot - somma_provvisoria -
        somma_provvisoria));
73
74                 break;

```

```

75         }
76         else
77         {
78             // Se il bit e' 1, lo riporto a 0 e continuo verso
              sinistra
79             apple_bit[i] = 0;
80         }
81     }
82 }
83
84 // Stampa della differenza minima trovata
85 cout << diff << endl;
86
87 return 0;
88 }

```

Si osserva che l'implementazione presentata può essere semplificata e resa più efficiente utilizzando una rappresentazione delle configurazioni tramite un'unica variabile intera (*bitmask*). In questo modo è possibile generare direttamente tutte le possibili suddivisioni con un ciclo che varia da 0 a $2^N - 1$, evitando la simulazione manuale dell'incremento binario mediante un vettore di booleani. Tale tecnica migliora la leggibilità del codice e riduce la complessità dell'implementazione. Un possibile esempio di questa soluzione è riportato di seguito; un'analisi più approfondita viene lasciata al lettore interessato.

```

1   for (int mask = 0; mask < (1 << N); mask++)
2   {
3       int64_t somma = 0;
4
5       for (int i = 0; i < N; i++)
6       {
7           if (mask & (1 << i))
8               somma += apples[i];
9       }
10
11       diff = min(diff, abs(somma_tot - 2 * somma));
12   }

```

Costo Computazionale

Il costo computazionale è:

- $O(n)$ per il ciclo for alla riga 56, che calcola la somma dei pesi delle N mele
- $O(n)$ per la funzione `somma_apple`, che scorre tutte le n mele alla riga 11.
- $O(2^n)$ per il ciclo che genera tutte le possibili bit mask (riga 65)

La complessità è:

$$\begin{aligned}
 &= O(n) + O(2^n) \cdot O(n) \\
 &= O(2^n \cdot n)
 \end{aligned}$$

dove:

- n è il numero di mele

Dai vincoli del problema:

$$1 \leq n \leq 20$$

il numero massimo di combinazioni da esplorare è:

$$2^{20} = 1\,048\,576$$

Sostituendo nella complessità, otteniamo:

$$O(2^n \cdot n) = O(2^{20} \cdot 20) \approx O(2 \cdot 10^7)$$

Considerando una macchina che esegue circa 10^8 operazioni al secondo, il tempo stimato di esecuzione è:

$$\frac{2 \cdot 10^7}{10^8} = 0.2 \text{ secondi}$$

Pertanto, questo approccio brute force risulta accettabile per i vincoli del problema e rientra nel time limit di 1 secondo.

4.3 Prefix Sum

La tecnica delle *Prefix Sum* (somme prefisse) è un metodo di preprocessing che consente di calcolare in modo efficiente la somma degli elementi di un array su un determinato intervallo. L'idea fondamentale consiste nel costruire un nuovo array in cui ogni elemento rappresenta la somma cumulativa degli elementi precedenti.

Supponiamo di avere un array di N interi del tipo

$$a_0, a_1, \dots, a_n$$

e di voler rispondere alla domanda: qual è la somma degli elementi da a_l ad a_r ? Con $0 \leq l \leq r < N$. In altre parole ci stiamo chiedendo qual è la somma dall'elemento l all'elemento r .

Per risolvere questo problema potremmo essere tentati di utilizzare una tecnica del tipo **brute force** che, in C++, potremmo codificare così:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main(){
5      int N;
6      cin >> N;
7
8      vector<int> v(N); // vettore di dimensione N
9
10     for(int i = 0; i < N; i++){
11         cin >> v[i];
12     }
13
14     int l, r;
15     cin >> l >> r; // si assume 0 <= l <= r < N
16
17     int somma = 0;
18
19     // brute force: somma degli elementi nell'intervallo [l, r]
```

```

20     for(int i = l; i <= r; i++){
21         somma += v[i];
22     }
23
24     cout << somma << endl;
25 }

```

Supponiamo di dover calcolare Q somme su intervalli $[l, r]$, differenti per ogni query Q_i . Qual è la complessità computazionale totale in questo caso?

Proviamo a scrivere il codice seguendo, ancora una volta, un approccio **brute force**.

```

1     #include <bits/stdc++.h>
2     using namespace std;
3
4     int main(){
5         int N;
6         cin >> N;
7
8         vector<int> v(N); // vettore di dimensione N
9         for(int i = 0; i < N; i++){
10             cin >> v[i];
11         }
12
13         int Q;
14         cin >> Q; // numero di query
15
16         for(int q = 0; q < Q; q++){
17             int l, r;
18             cin >> l >> r; // si assume 0 <= l <= r < N
19
20             int somma = 0;
21             for(int i = l; i <= r; i++){
22                 somma += v[i]; // brute force: somma degli elementi
23                                 // nell'intervallo [l, r]
24             }
25
26             cout << somma << endl;
27         }
28
29         return 0;
30     }

```

Costo Computazionale:

Il costo computazionale dell'approccio brute force è il seguente:

- $O(Q)$ per il ciclo esterno sulle Q query
- $O(N)$ per il ciclo interno che calcola la somma nell'intervallo $[l, r]$ nel caso peggiore

Pertanto, la complessità computazionale totale è:

$$O(Q) \cdot O(N) = O(Q \cdot N)$$

Questo approccio, sebbene semplice e intuitivo, non è ottimale per valori grandi di N e Q .

La soluzione più efficiente utilizza la tecnica delle **somme prefisse** (prefix sum). L'idea consiste nel creare un secondo vettore S di dimensione N in cui ogni elemento contiene la somma cumulativa degli elementi dell'array originale v fino a quella posizione:

$$S[i] = v[0] + v[1] + \dots + v[i] \quad \text{per } i = 0, 1, \dots, N-1$$

In questo modo, la somma di un intervallo qualsiasi $[l, r]$ può essere calcolata in tempo costante tramite:

$$\sum_{i=l}^r v[i] = \begin{cases} S[r], & \text{se } l = 0 \\ S[r] - S[l-1], & \text{se } l > 0 \end{cases}$$

In altre parole, una volta costruito il vettore S (con complessità $O(N)$), per calcolare una qualsiasi somma da l ad r non faccio altro che eseguire il seguente calcolo:

$$S[r] - S[l-1] \tag{4.1}$$

Quindi, ad esempio, per calcolare la somma degli elementi dall'indice 3 all'indice 6, basta prendere la somma cumulativa di tutti gli elementi fino all'indice 6 e sottrarre la somma cumulativa fino all'indice 2. In questo modo si ottiene direttamente la somma degli elementi dall'elemento 3 all'elemento 6, estremi inclusi.

Come detto, il preprocessing per costruire l'array delle somme prefisse richiede $O(N)$, mentre ogni query può essere risolta in $O(1)$. Pertanto, la complessità totale diventa:

$$O(N) + O(Q) = O(N + Q)$$

Questa soluzione è estremamente più efficiente rispetto al brute force quando sia N che Q sono grandi.

Proviamo a codificare il problema:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main(){
5      int N;
6      cin >> N;
7
8      vector<int> v(N); // vettore di dimensione N
9      for(int i = 0; i < N; i++){
10         cin >> v[i];
11     }
12
```

```

13     vector<int> prefix(N); // vettore delle somme prefisse
14     prefix[0] = v[0];      // il primo elemento lo stesso di v
15                               [0]
16
17     // costruisco le somme prefisse
18     for(int i = 1; i < N; i++){
19         prefix[i] = v[i] + prefix[i-1]; // sommo l'elemento
20                                         corrente v[i] con la somma precedente
21     }
22
23     // Una volta costruito prefix, la somma di un intervallo [l, r]
24     // si calcola in O(1)
25     int Q;
26     cin >> Q;
27
28     for(int q = 0; q < Q; q++){
29         int l, r;
30         cin >> l >> r;
31
32         int somma;
33         if(l == 0)
34             somma = prefix[r]; // se l = 0, la somma e' prefix[r]
35         else
36             somma = prefix[r] - prefix[l-1]; // altrimenti sottraggo
37                                         prefix[l-1]
38
39         cout << somma << endl;
40     }
41
42     return 0;
43 }

```

Costo Computazionale:

Il costo computazionale è il seguente:

- $O(N)$ per riempire il vettore *prefix* (riga 17)
- $O(Q)$ per il ciclo esterno sulle Q query
- $O(1)$ per il calcolo di *somma*

Pertanto, la complessità computazionale totale è:

$$O(N) + O(Q) = O(N + Q)$$

Rispetto all'approccio *brute force* (come mostrato in precedenza), questo metodo permette un notevole risparmio di tempo quando Q ed N sono molto grandi.

4.3.1 C - Index $\times$ A(Continuous ver.)

Link al problema

Time limit = 1.00s, Memory limit = 512 MB

Problem Statement

You are given an integer sequence $A = (A_1, A_2, \dots, A_N)$ of length N . Find the maximum value of $\sum_{i=1}^M i \times B_i$ for a contiguous subarray $B = (B_1, B_2, \dots, B_M)$ of A of length M .

Notes

A contiguous subarray of a number sequence is a sequence that is obtained by removing 0 or more initial terms and 0 or more final terms from the original number sequence.

For example, $(2, 3)$ and $(1, 2, 3)$ are contiguous subarrays of $(1, 2, 3, 4)$, but $(1, 3)$ and $(3, 2, 1)$ are not contiguous subarrays of $(1, 2, 3, 4)$.

Constraints

- $1 \leq M \leq N \leq 2 \cdot 10^5\ 000$
- $2 \cdot 10^5 \leq A_i \leq 2 \cdot 10^5\ 000\ 000$
- All values in input are integer.

Input

Input is given from Standard Input in the following format:

$N\ M$
 $A_1\ A_2\ \dots\ A_N$

Output

Print the answer.

Sample Input1

4 2
5 4 -1 8

Sample Output1

15

When $B = (A_3, A_4)$, we have $\sum_{i=1}^M i \cdot B_i = 1 \cdot (-1) + 2 \cdot 8 = 15$. Since it is impossible to achieve 16 or a larger value, the solution is 15.

Note that you are not allowed to choose, for instance, $B = (A_1, A_4)$.

Sample Input2

10 4
-3 1 -4 1 -5 0 -2 6 -5 3

Sample Output2

31

Idea (prefix)

Il problema richiede di calcolare la seguente somma:

$$S = \sum_{i=1}^M i \cdot B_i \quad (4.2)$$

in modo tale che S sia massima, dove $B = (B_1, B_2, \dots, B_M)$ è un **sotto-array contiguo** della sequenza A .

Un approccio intuitivo consiste nei seguenti passi:

1. Consideriamo una **finestra di lunghezza** M che scorre lungo l'array A . Denotiamo questa finestra come B inizialmente posizionata sugli indici $0, \dots, M-1$.
2. Per la finestra corrente, calcoliamo la sommatoria richiesta:

$$S = \sum_{i=1}^M i \cdot B_i \quad (4.3)$$

3. Spostiamo la finestra di **un passo verso destra** (cioè agli indici $1, \dots, M$) e ricalcoliamo la sommatoria S . Ad ogni passo, confrontiamo il valore corrente con il massimo trovato finora e aggiorniamo il massimo se necessario.
4. Ripetiamo questo procedimento per tutte le finestre contigue di lunghezza M presenti in A .

Alla fine, il **massimo valore** ottenuto tra tutte le finestre corrisponde alla soluzione del problema.

Come possiamo calcolare la sommatoria richiesta? Ricalcolarla da zero per ogni finestra sarebbe molto dispendioso in termini computazionali (lasciamo al lettore l'implementazione di questo metodo). Perciò, dobbiamo trovare una strategia più efficiente.

Possiamo osservare che:

$$S = \sum_{i=1}^M i \cdot B_i = \sum_{j=left}^{right} (j - left + 1) \cdot A[j] \quad (4.4)$$

dove con $left$ indichiamo l'indice del limite sinistro della finestra e con $right$ l'indice del limite destro.

Espandendo i termini della sommatoria, si può notare che la quantità all'ultimo membro della equazione 4.4 è proprio S .

Infatti, considerando che j parta da $left$ e finisca a $right = left + M - 1$, S diventa:

$$\begin{aligned} S &= \sum_{j=left}^{right} (j - left + 1) \cdot A[j] \\ &= (left - left + 1) \cdot A[left] + (left + 1 - left + 1) \cdot A[left + 1] + \\ &\quad + (left + 2 - left + 1) \cdot A[left + 2] + \dots \\ &\quad + (left + M - 1 - left + 1) \cdot A[left + M - 1] \end{aligned} \quad (4.5)$$

Ovvero, in forma compatta:

$$S = 1 \cdot A[left] + 2 \cdot A[left + 1] + \dots + M \cdot A[right] \quad (4.6)$$

Che è esattamente la quantità che vogliamo massimizzare.

Ripartendo quindi dalla equazione 4.4, possiamo riarrangiarla in questo modo

$$\begin{aligned} S &= \sum_{j=left}^{right} (j - left + 1) \cdot A[j] = \\ &= \sum_{j=left}^{right} j \cdot A[j] + (left - 1) \cdot \sum_{j=left}^{right} A[j] \end{aligned} \quad (4.7)$$

Da qui possiamo notare due contributi importanti:

- $\sum_{j=left}^{right} A[j]$ (contributo destro)
- $\sum_{j=left}^{right} j \cdot A[j]$ (contributo sinistro)

Creiamo quindi due vector prefix tali per cui l'i-esimo elemento è:

- $prefix[i] = A[0] + A[1] + \dots + A[i]$, per $i = 0, 1, \dots, N - 1$
- $w_prefix[i] = 0 \cdot A[0] + 1 \cdot A[1] + \dots + i \cdot A[i]$, per $i = 0, 1, \dots, N - 1$

Dopo aver costruito i due vettori, possiamo calcolare i due contributi in $O(1)$:

Contributo destro

$$\sum_{j=left}^{right} A[j] = \begin{cases} prefix[right], & \text{se } left = 0, \\ prefix[right] - prefix[left - 1], & \text{se } left > 0 \end{cases}$$

Contributo sinistro

$$\sum_{j=left}^{right} j \cdot A[j] = \begin{cases} w_prefix[right], & \text{se } left = 0, \\ w_prefix[right] - w_prefix[left - 1], & \text{se } left > 0 \end{cases}$$

Implementazione C++:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int N, M;
7      cin >> N >> M; // Leggiamo la lunghezza dell'array e la
                        // dimensione della finestra
8
9      vector<int64_t> A(N);
10     for (int i = 0; i < N; i++)
11     {
12         cin >> A[i]; // Leggiamo gli elementi dell'array
```

```

13     }
14
15     // COSTRUZIONE DEL VETTORE prefix
16     // prefix[i] = somma dei primi i+1 elementi di A
17     vector<int64_t> prefix(N);
18     prefix[0] = A[0]; // il primo elemento della somma cumulativa
        A[0]
19
20     for (int i = 1; i < N; i++)
21     {
22         prefix[i] = A[i] + prefix[i - 1]; // somma cumulativa fino
            all'indice i
23     }
24
25     // COSTRUZIONE DEL VETTORE w_prefix
26     // w_prefix[i] = somma pesata dei primi i+1 elementi: 0*A[0] +
        1*A[1] + ... + i*A[i]
27     vector<int64_t> w_prefix(N);
28     w_prefix[0] = 0; // il primo elemento pesato      0*A[0]
29
30     for (int i = 1; i < N; i++)
31     {
32         w_prefix[i] = i * A[i] + w_prefix[i - 1]; // somma pesata
            cumulativa fino all'indice i
33     }
34
35     int left = 0; // limite sinistro della finestra
36     int64_t ans = INT64_MIN; // variabile per salvare il
        massimo valore trovato
37     int64_t current_value; // valore della sommatoria per la
        finestra corrente
38
39     // Scorriamo l'array per calcolare tutte le finestre di
        lunghezza M
40     for (int right = left + M - 1; right < N; right++)
41     {
42         // Assicuriamoci che la finestra sia lunga esattamente M
43         while (right - left > M - 1)
44         {
45             left++; // spostiamo il limite sinistro verso destra
46         }
47
48         // CALCOLO DELLA SOMMA PESATA NELLA FINESTRA [left, right]
49         if (left == 0)
50         {
51             // Se la finestra parte dall'inizio dell'array
52             // allora la sommatoria pesata e' semplicemente
                w_prefix[right]
53             // e aggiungiamo prefix[right] per correggere l'
                indicizzazione
54             current_value = w_prefix[right] + prefix[right];
55         }
56         else

```

```

57     {
58         // Se la finestra parte da left > 0
59         // Usiamo la formula generale per le somme pesate di
           una finestra
60         current_value = (w_prefix[right] - w_prefix[left - 1])
61         - (left - 1) * (prefix[right] - prefix[left - 1]);
62     }
63
64     // Aggiorniamo il massimo trovato
65     ans = max(current_value, ans);
66 }
67
68 // Stampa del risultato finale
69 cout << ans;
70
71 return 0;
72 }

```